Experiment No. 1.a

Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

```python
import matplotlib.pyplot as plt
import pandas as pd

data = {
    "Year": [2015, 2016, 2017, 2018, 2019, 2020],
    "Jobs": [5000, 8000, 12000, 20000, 30000, 40000],
}

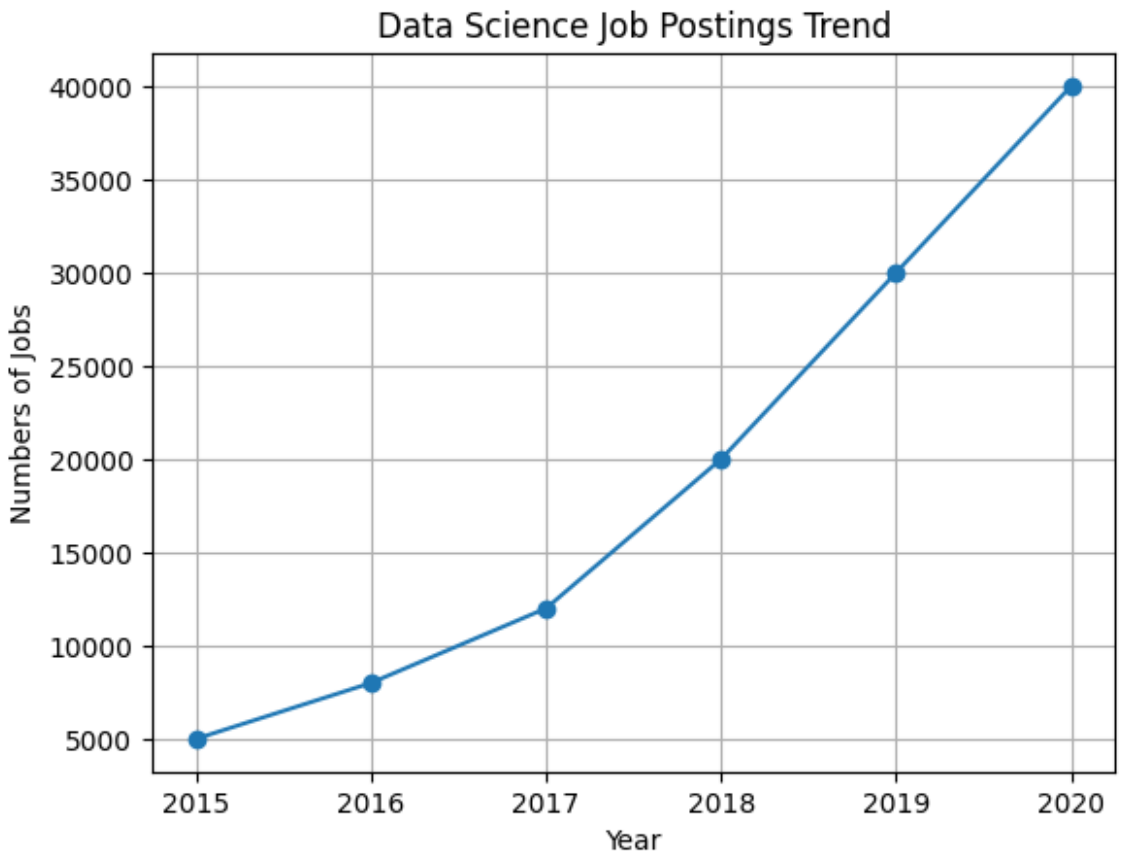
df = pd.DataFrame(data)
print(df)

plt.plot(df["Year"], df["Jobs"], marker="o")
plt.title("Data Science Job Postings Trend")
plt.xlabel("Year")
plt.ylabel("Numbers of Jobs")
plt.grid()
plt.show()
```

```
   Year   Jobs
0  2015   5000
1  2016   8000
2  2017  12000
3  2018  20000
4  2019  30000
5  2020  40000
```

Experiment No. 1.b

Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.




```
import matplotlib.pyplot as plt
import pandas as pd

import pandas as pd

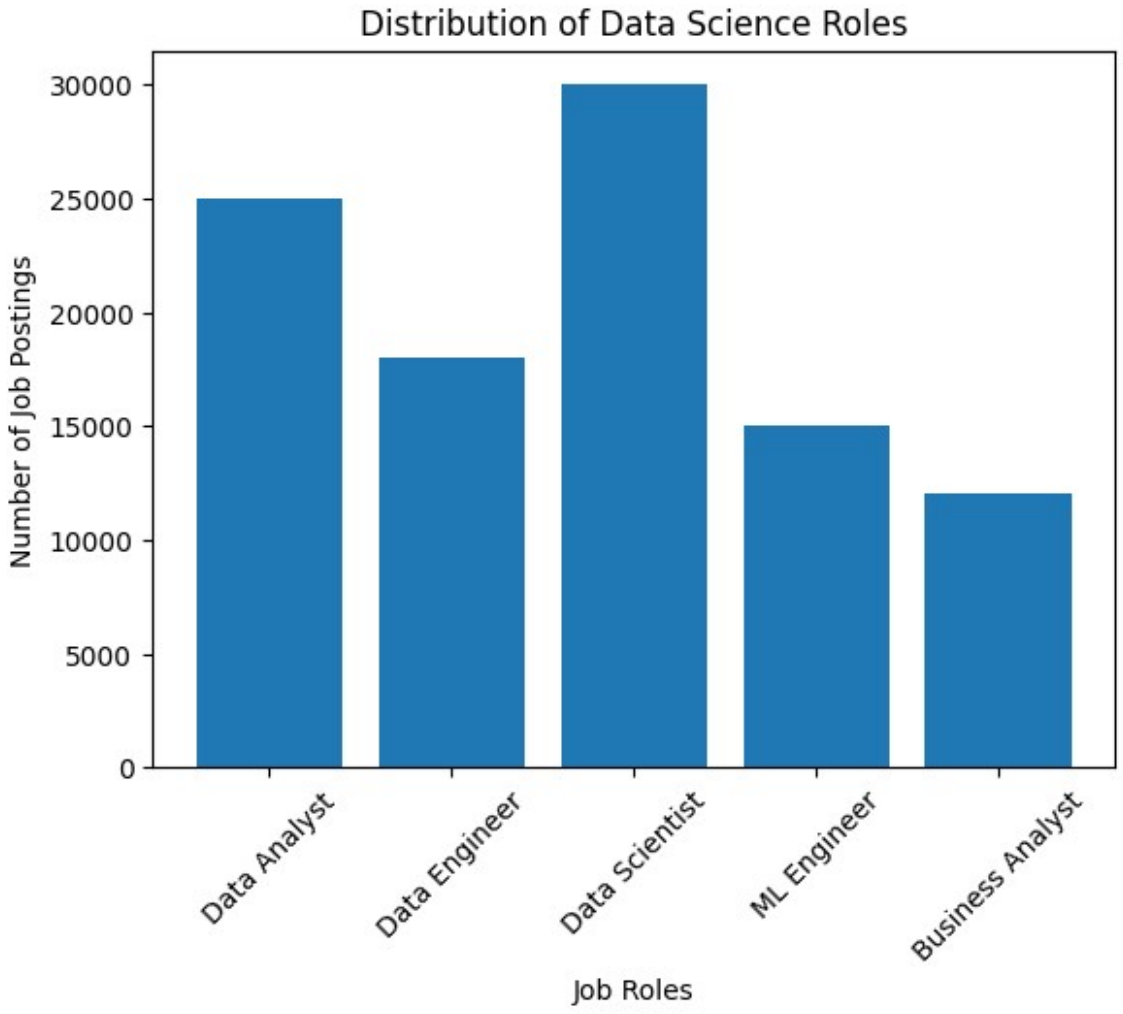
data = {
    "Role": [
        "Data Analyst",
        "Data Engineer",
        "Data Scientist",
        "ML Engineer",
        "Business Analyst",
    ],
    "Number_of_Jobs": [25000, 18000, 30000, 15000, 12000],
}

df = pd.DataFrame(data)
print(df)

plt.bar(df["Role"], df["Number_of_Jobs"])
plt.title("Distribution of Data Science Roles")
plt.xlabel("Job Roles")
plt.ylabel("Number of Job Postings")
```

```
plt.xticks(rotation=45)
plt.show()
```

	Role	Number_of_Jobs
0	Data Analyst	25000
1	Data Engineer	18000
2	Data Scientist	30000
3	ML Engineer	15000
4	Business Analyst	12000




```
import pandas as pd

# Structured Data
structured = pd.DataFrame({"Name": ["John", "Alice"], "Age": [25,
30]})

print("Structured Data")
print(structured)
```

Experiment No. 1.c

Conduct an experiment to differentiate Structured, Un-structured and Semi-structured data based on data sets given.

```python
# Semi-structured Data
semi_structured = {"Name": "Bob", "Age": 28, "Skill": "Python"}

print("\nSemi-structured Data:")
print(semi_structured)

# Unstructured Data
unstructured = "Data Science uses machine learning and AI"

print("\nUnstructured Data:")
print(unstructured)
```

```
Structured Data
    Name  Age
0   John   25
1  Alice   30

Semi-structured Data:
{'Name': 'Bob', 'Age': 28, 'Skill': 'Python'}

Unstructured Data:
Data Science uses machine learning and AI
```

Experiment No. 1.d

Conduct an experiment to encrypt and decrypt given sensitive data. Use the cryptography library Fernet.

```
!pip install cryptography
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)

data = "My password is 12345"

encrypted_data = cipher.encrypt(data.encode())
print("Encrypted data:")
print(encrypted_data)

decrypted_data = cipher.decrypt(encrypted_data).decode()

print("\nDecrypted Data:")
print(decrypted_data)
```

```
Decrypted Data:
My password is 12345
```

Experiment No. 2

Upload and analyze the data set given in CSV format and perform data preprocessing and visualization.

```
import matplotlib.pyplot as plt
import pandas as pd

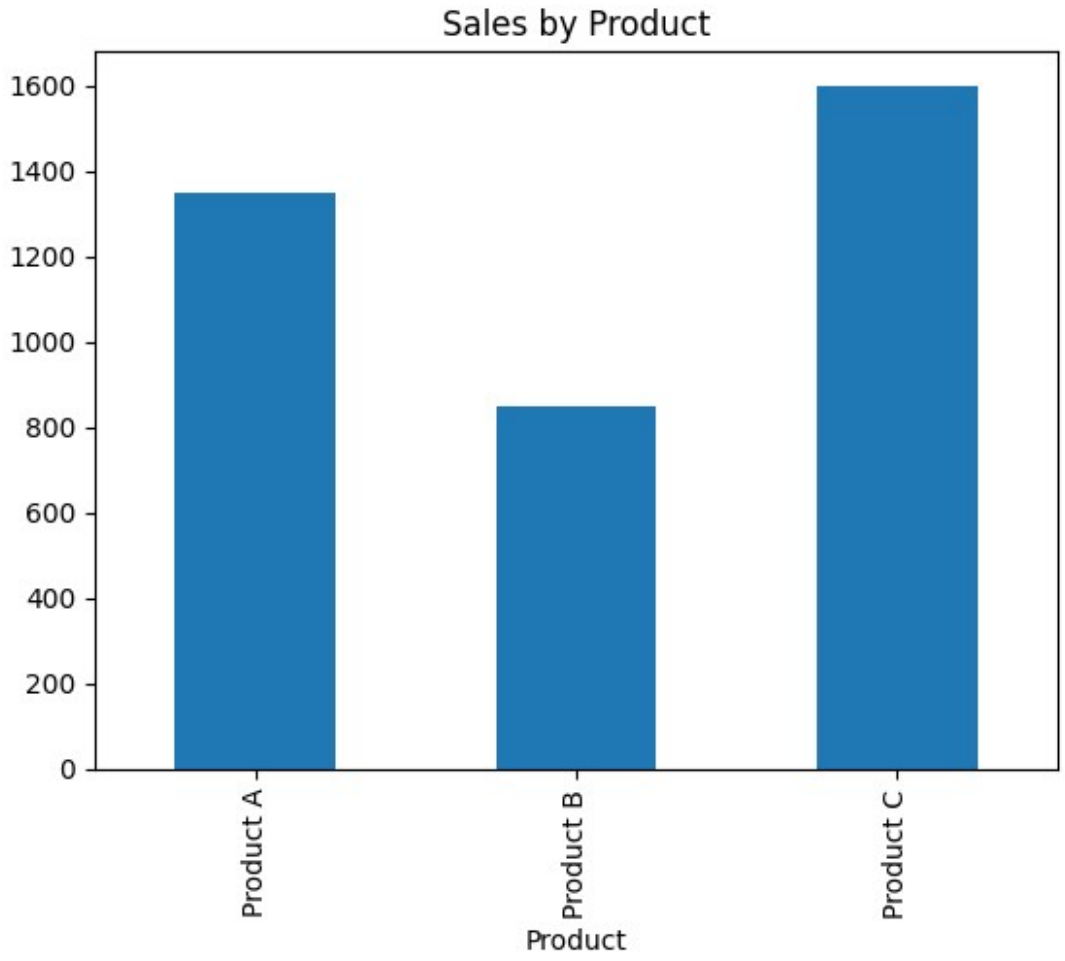
df = pd.read_excel('/content/sales_data.xlsx')
print(df)
```

```
                  Date    Product  Sales  Quantity Region
0   2023-01-01 00:00:00  Product A    200         4  North
1   2023-02-01 00:00:00  Product B    150         3  South
2   2023-03-01 00:00:00  Product A    220         5  North
3   2023-04-01 00:00:00  Product C    300         6   East
4   2023-05-01 00:00:00  Product B    180         4   West
5   2023-06-01 00:00:00  Product A    210         5  North
6   2023-07-01 00:00:00  Product C    320         7   East
7   2023-08-01 00:00:00  Product B    160         3  South
8   2023-09-01 00:00:00  Product A    230         6  North
9   2023-10-01 00:00:00  Product C    310         7   East
10  2023-11-01 00:00:00  Product B    190         4   West
11  2023-12-01 00:00:00  Product A    240         6  North
12           13-01-2023  Product C    330         8   East
13           14-01-2023  Product B    170         3  South
14           15-01-2023  Product A    250         7  North
15           16-01-2023  Product C    340         8   East
```

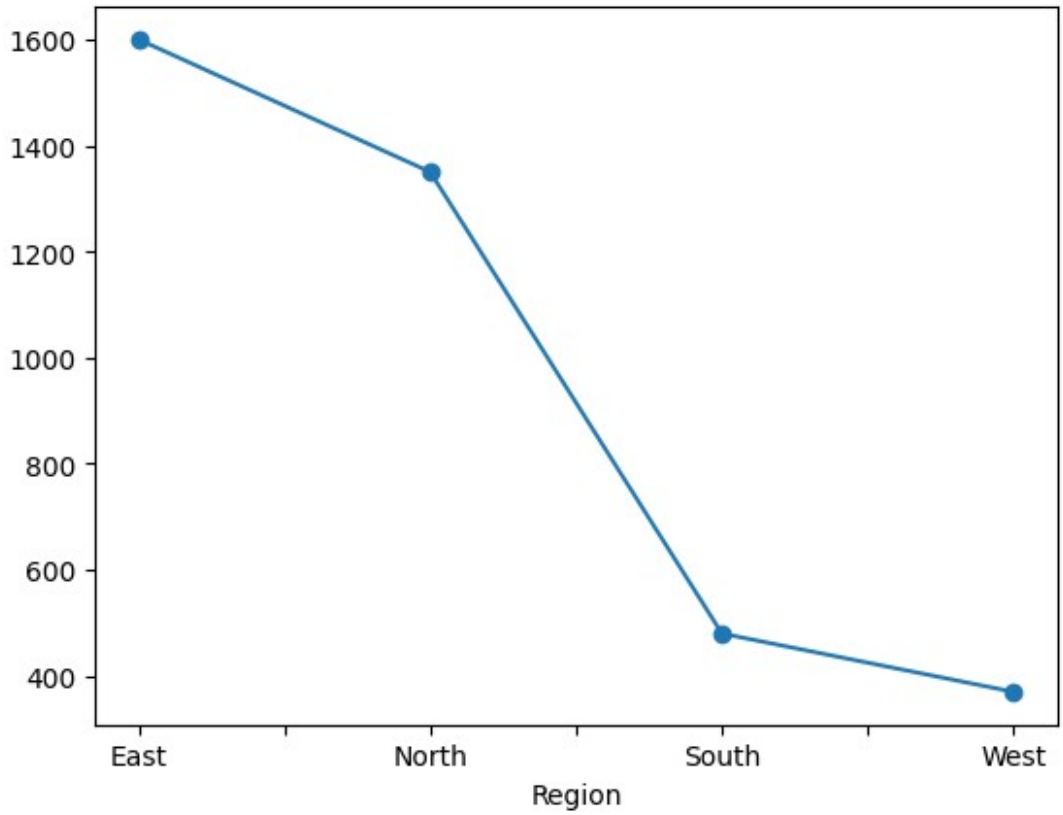
```
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
df = df.drop_duplicates().dropna()
print(df)
```

```
         Date    Product  Sales  Quantity Region
0  2023-01-01  Product A    200         4  North
1  2023-02-01  Product B    150         3  South
2  2023-03-01  Product A    220         5  North
3  2023-04-01  Product C    300         6   East
4  2023-05-01  Product B    180         4   West
5  2023-06-01  Product A    210         5  North
6  2023-07-01  Product C    320         7   East
7  2023-08-01  Product B    160         3  South
8  2023-09-01  Product A    230         6  North
9  2023-10-01  Product C    310         7   East
10 2023-11-01  Product B    190         4   West
11 2023-12-01  Product A    240         6  North
12 2023-01-13  Product C    330         8   East
13 2023-01-14  Product B    170         3  South
14 2023-01-15  Product A    250         7  North
15 2023-01-16  Product C    340         8   East
```

```
df.groupby('Product')['Sales'].sum().plot(kind='bar')
plt.title('Sales by Product')
plt.show()
```




```
df.groupby('Region')['Sales'].sum().plot(kind='line', marker='o')
plt.show()
```


1600
1400
1200
1000
800
600
400
East
North
South
West
Region

Experiment No. 3.a

Conduct an experiment to show data visualization using line plot.

Description: Take any sample data either through CSV file or data fetched directly through code.

Experiment No. 3.b

Conduct an experiment to show data visualization using bar chart.

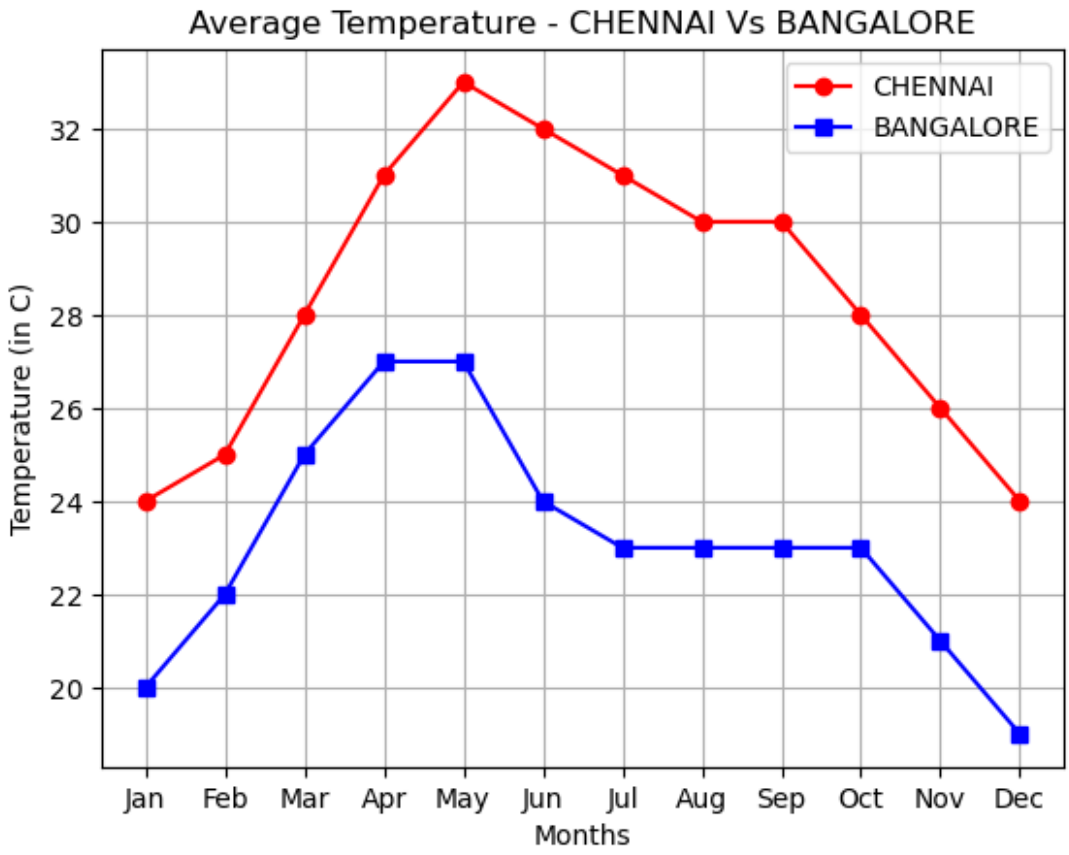
Description: Take any sample data either through CSV file or data fetched directly through code.

Experiment No. 3.c

Conduct an experiment to show data visualization using pie chart.

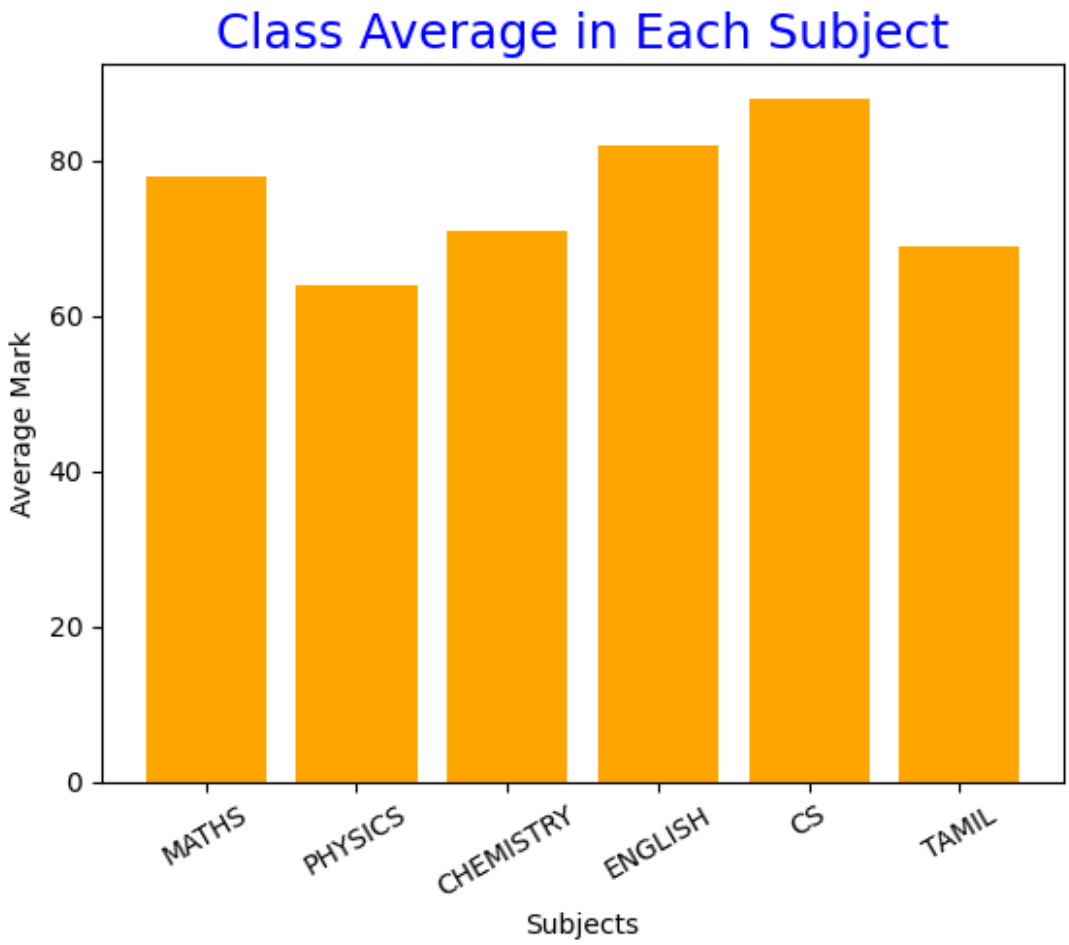
Description: Take any sample data either through CSV file or data fetched directly through code.

```
import matplotlib.pyplot as rain
Months=['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','N
ov','Dec']
Chennai=[24,25,28,31,33,32,31,30,30,28,26,24]
Bangalore=[20,22,25,27,27,24,23,23,23,23,21,19]
rain.plot(Months,Chennai,color="red",marker="o",label="CHENNAI")
rain.plot(Months,Bangalore,color="blue",marker="s",label="BANGALORE")
rain.title("Average Temperature - CHENNAI Vs BANGALORE")
rain.xlabel("Months")
rain.ylabel("Temperature (in C)")
rain.legend(loc="upper right")
rain.grid(True)
rain.show()
```



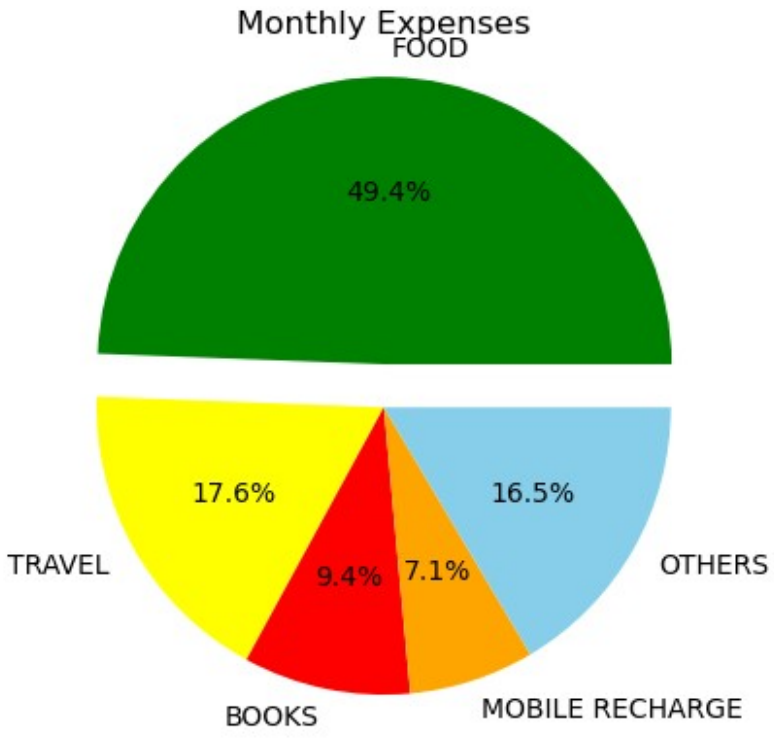

```
import matplotlib.pyplot as marks
import numpy as np
Subjects=['MATHS','PHYSICS','CHEMISTRY','ENGLISH','CS','TAMIL']
xaxis=np.arange(len(Subjects))
Average_mark=[78,64,71,82,88,69]
marks.bar(Subjects,Average_mark,color="orange")
marks.xticks(xaxis,Subjects,rotation=30)
marks.xlabel("Subjects")
```

```
marks.ylabel("Average Mark")
marks.title("Class Average in Each Subject",fontsize=18,color="blue")
marks.show()
```




```
import matplotlib.pyplot as spend

labels=['FOOD','TRAVEL','BOOKS','MOBILE RECHARGE','OTHERS']
Amount=[4200,1500,800,600,1400]
colors=['green','yellow','red','orange','skyblue']
explode=(0.15,0,0,0,0)
spend.pie(Amount,labels=labels,colors=colors,explode=explode,autopct='
%0.1f%%')
spend.title('Monthly Expenses')
spend.show()
```


Monthly Expenses
FOOD
49.4%
17.6%
16.5%
TRAVEL
9.4%
7.1%
OTHERS
BOOKS
MOBILE RECHARGE

Experiment No. 4

To count the frequency of occurrence of a word in a body of text, which is often needed during text processing.

Description: Import the word_tokenize function and Gutenberg corpus.

```python
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
nltk.download('gutenberg')
nltk.download('punkt')
nltk.download('punkt_tab')
sample=gutenberg.raw("carroll-alice.txt")
token=word_tokenize(sample)
wlist=[]
for i in range(60):
    wlist.append(token[i])
wordfreq=[wlist.count(w) for w in wlist]
print("Pairs\n"+str(list(zip(wlist,wordfreq))))

from collections import Counter
words=[w.lower() for w in token if w.isalpha()]
print("\nTop 10 words in whole text")
print(Counter(words).most_common(10))
```

```
Pairs
[('[', 1), ('Alice', 2), ("'s", 1), ('Adventures', 1), ('in', 1),
('Wonderland', 1), ('by', 2), ('Lewis', 1), ('Carroll', 1), ('1865',
1), (']', 1), ('CHAPTER', 1), ('I', 1), ('.', 1), ('Down', 1), ('the',
3), ('Rabbit-Hole', 1), ('Alice', 2), ('was', 2), ('beginning', 1),
('to', 2), ('get', 1), ('very', 1), ('tired', 1), ('of', 2),
('sitting', 1), ('by', 2), ('her', 2), ('sister', 2), ('on', 1),
('the', 3), ('bank', 1), (',', 2), ('and', 1), ('of', 2), ('having',
1), ('nothing', 1), ('to', 2), ('do', 1), (':', 1), ('once', 1),
('or', 2), ('twice', 1), ('she', 1), ('had', 2), ('peeped', 1),
('into', 1), ('the', 3), ('book', 1), ('her', 2), ('sister', 2),
('was', 2), ('reading', 1), (',', 2), ('but', 1), ('it', 1), ('had',
2), ('no', 1), ('pictures', 1), ('or', 2)]

Top 10 words in whole text
[('the', 1616), ('and', 810), ('to', 720), ('a', 631), ('she', 545),
('i', 543), ('it', 540), ('of', 499), ('said', 462), ('alice', 397)]

[nltk_data] Downloading package gutenberg to /home/c-s-
[nltk_data]     devanathan/nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to /home/c-s-
[nltk_data]     devanathan/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /home/c-s-
[nltk_data]     devanathan/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Experiment No. 5

Data Collection and Initial Exploration.

Objective: To collect, load, and perform initial exploration of the diabetes dataset.

```
import pandas as pd
db = pd.read_csv("/content/diabetes_dataset.csv")
print(db.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin
BMI  \
0            6      148             72             35        0  33.6

1            1       85             66             29        0  26.6

2            8      183             64              0        0  23.3

3            1       89             66             23       94  28.1

4            0      137             40             35      168  43.1

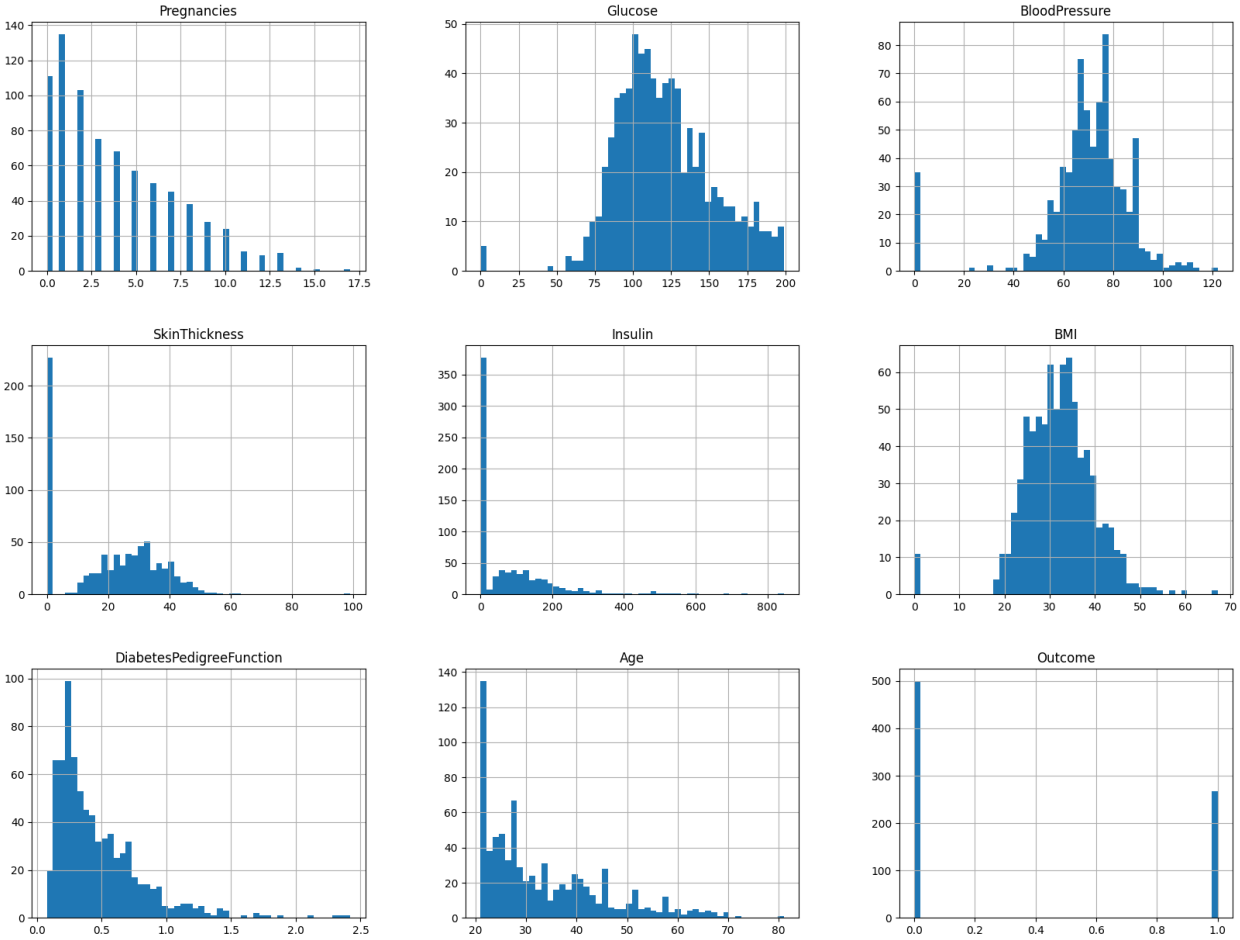

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

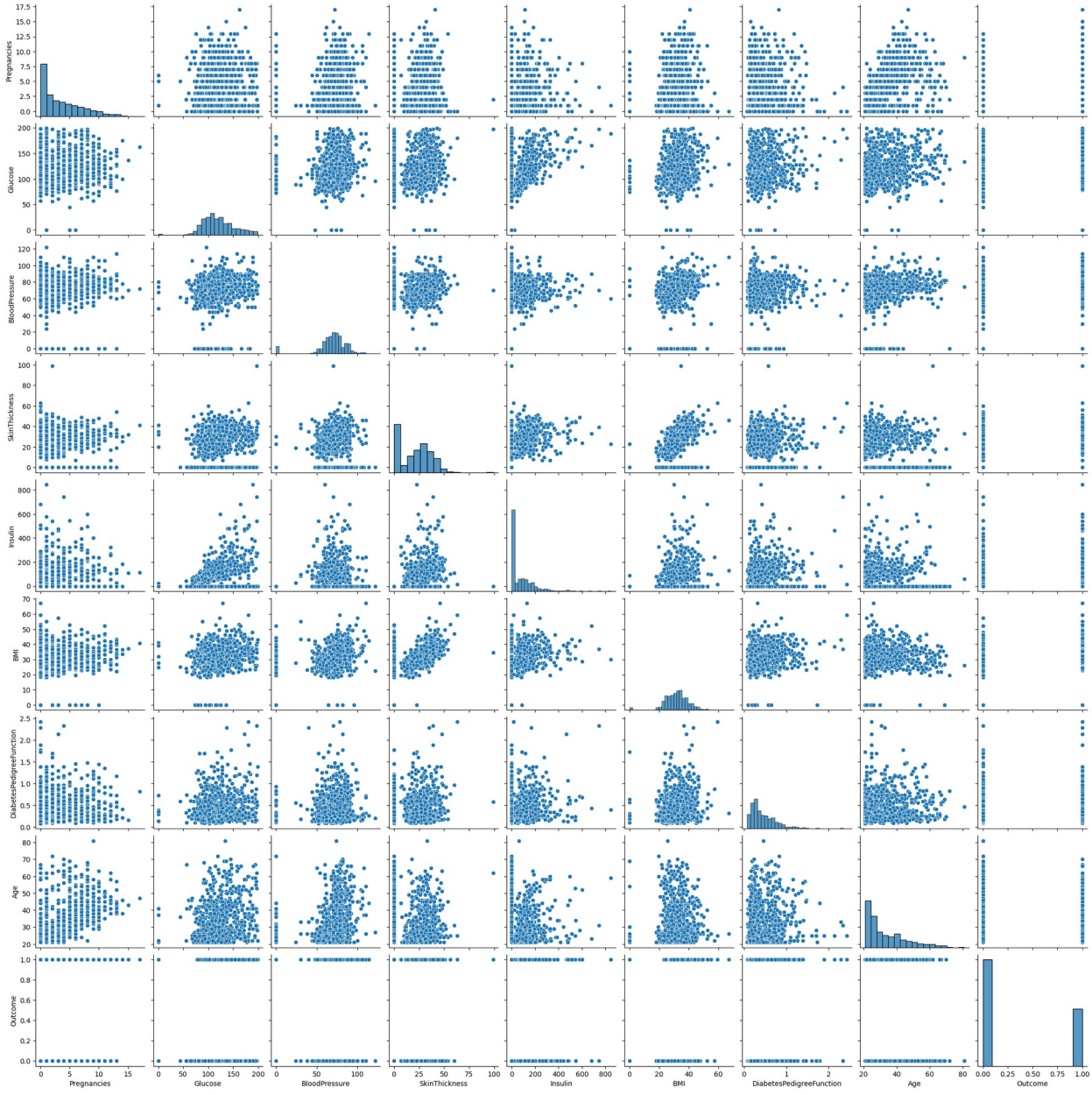
```
print(db.info())
print(db.describe())
import matplotlib.pyplot as plt
import seaborn as sns
db.hist(bins=50, figsize=(20, 15))
plt.show()
sns.pairplot(db)
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness
```

```
Insulin  \
count   768.000000  768.000000     768.000000     768.000000
768.000000
mean      3.845052  120.894531      69.105469      20.536458
79.799479
std       3.369578   31.972618      19.355807      15.952218
115.244002
min       0.000000    0.000000       0.000000       0.000000
0.000000
25%       1.000000   99.000000      62.000000       0.000000
0.000000
50%       3.000000  117.000000      72.000000      23.000000
30.500000
75%       6.000000  140.250000      80.000000      32.000000
127.250000
max      17.000000  199.000000     122.000000      99.000000
846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

Experiment No. 6

Time Series Analysis of Website Traffic.

Objective: To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

data = pd.read_csv('website_traffic.csv', parse_dates=['Date'],
index_col='Date')

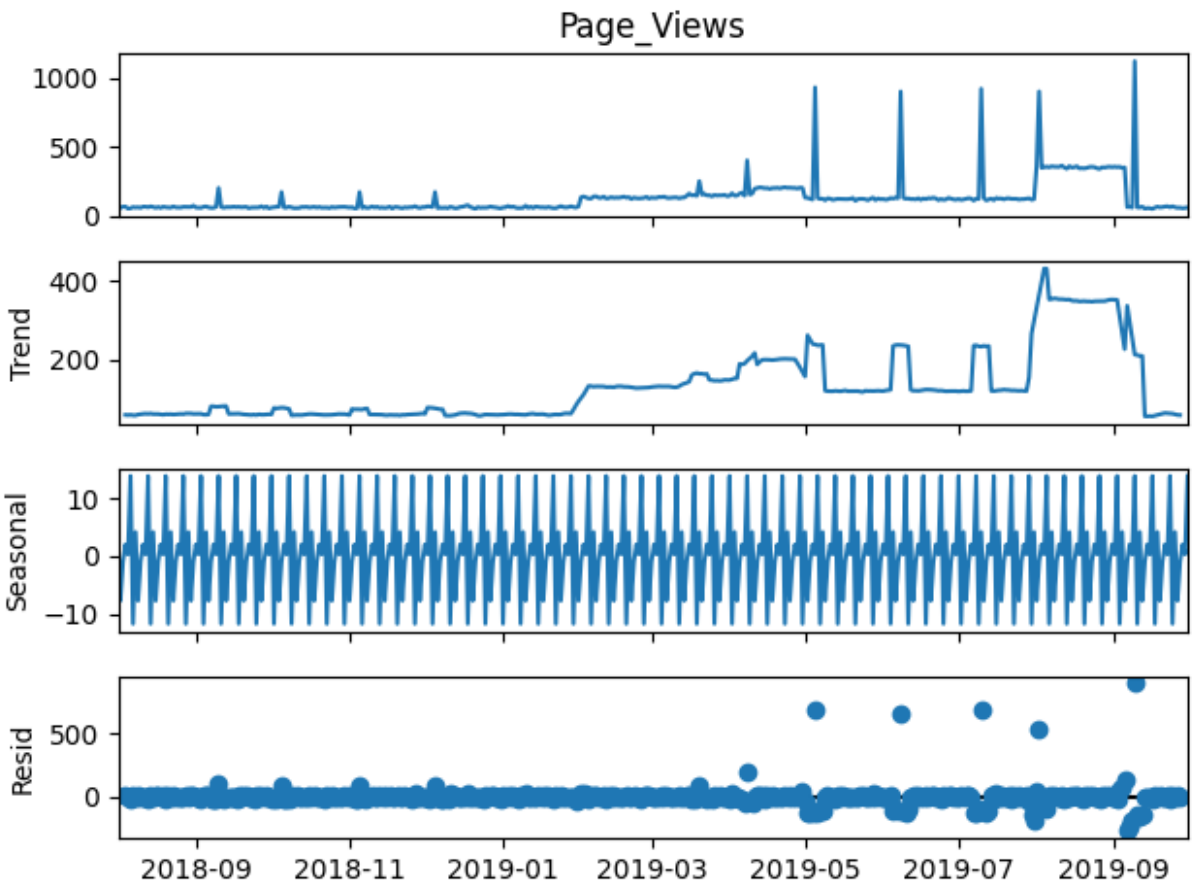
decomposition = seasonal_decompose(data['Page_Views'],
model='additive')
decomposition.plot()
plt.show()

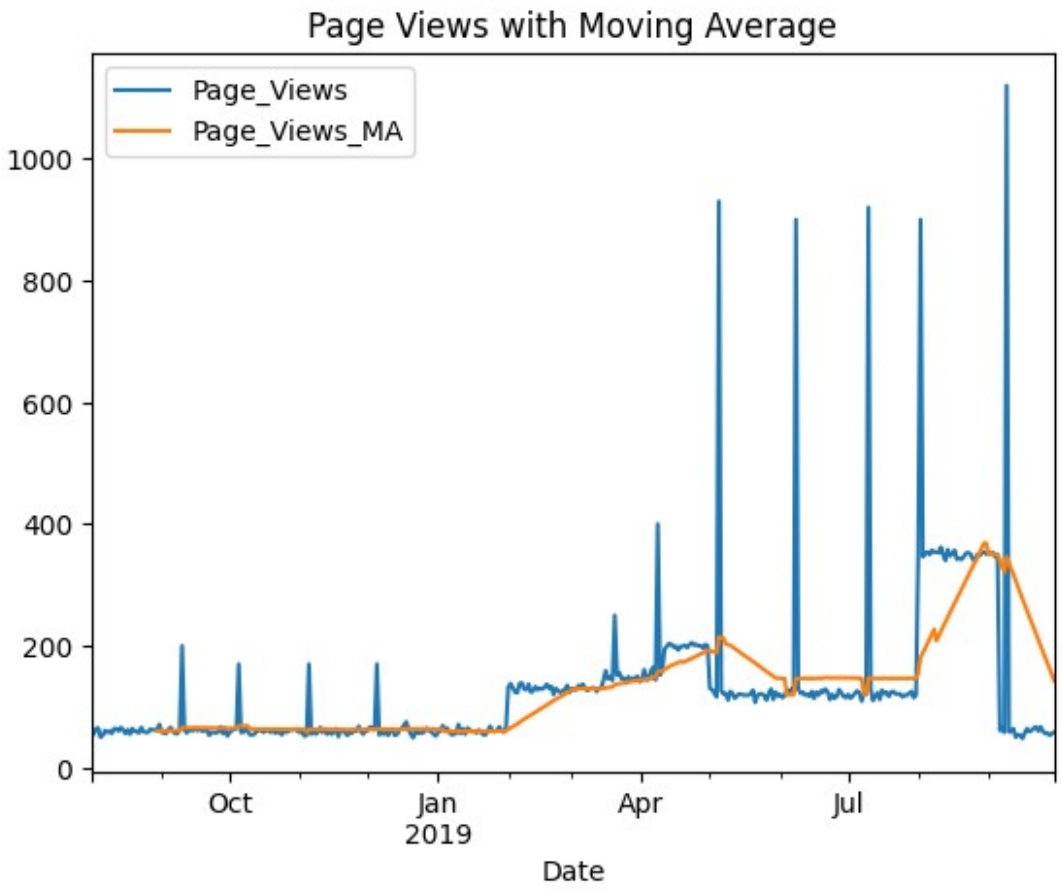
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

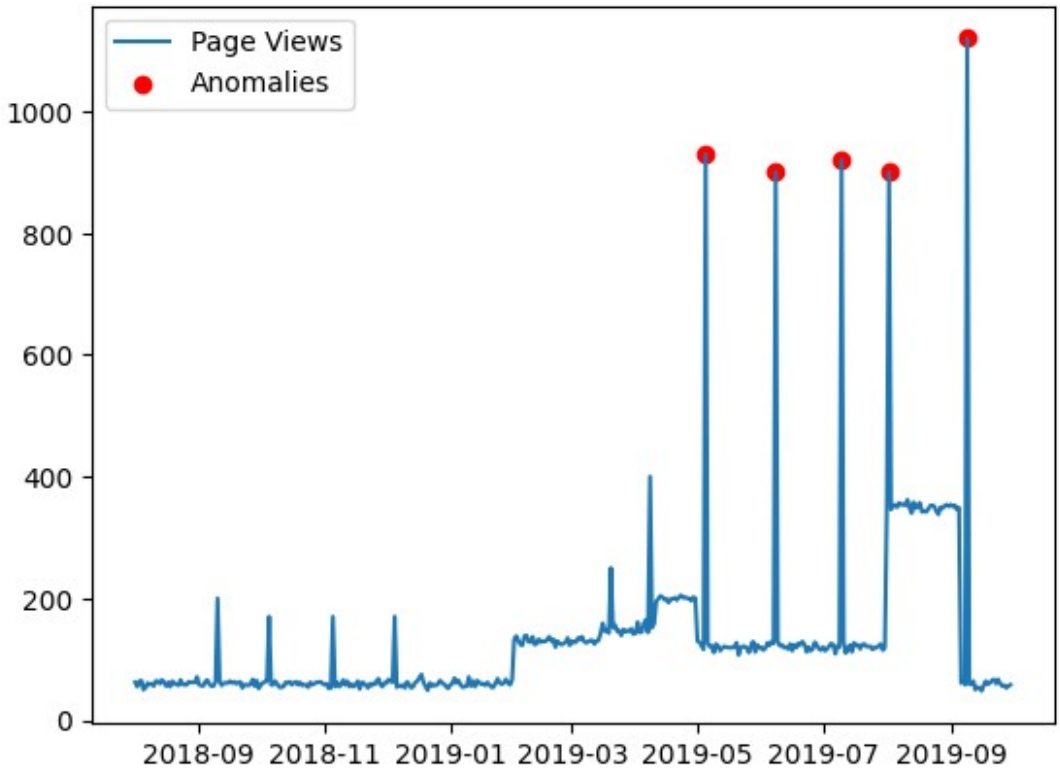
data['Page_Views_Z'] = (data['Page_Views'] -
data['Page_Views'].mean()) / data['Page_Views'].std()

anomalies = data[data['Page_Views_Z'].abs() > 3]

plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red',
label='Anomalies')
plt.legend()
plt.show()
```


Page_Views
1000
500
0
Trend
400
200
Seasonal
10
0
−10
Resid
500
0
2018-09
2018-11
2019-01
2019-03
2019-05
2019-07
2019-09


Page Views with Moving Average
Page_Views
Page_Views_MA
1000
800
600
400
200
0
Oct
Jan
2019
Apr
Jul
Date


Page Views
Anomalies
1000
800
600
400
200
0
2018-09
2018-11
2019-01
2019-03
2019-05
2019-07
2019-09

Experiment No. 7

Random Sampling and Sampling Distribution.

Objective: To explore random sampling from a population and understand the concept of sampling distribution.

```python
import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(population_mean, population_std,
population_size)

sample_sizes = [30, 50, 100]
num_samples = 1000

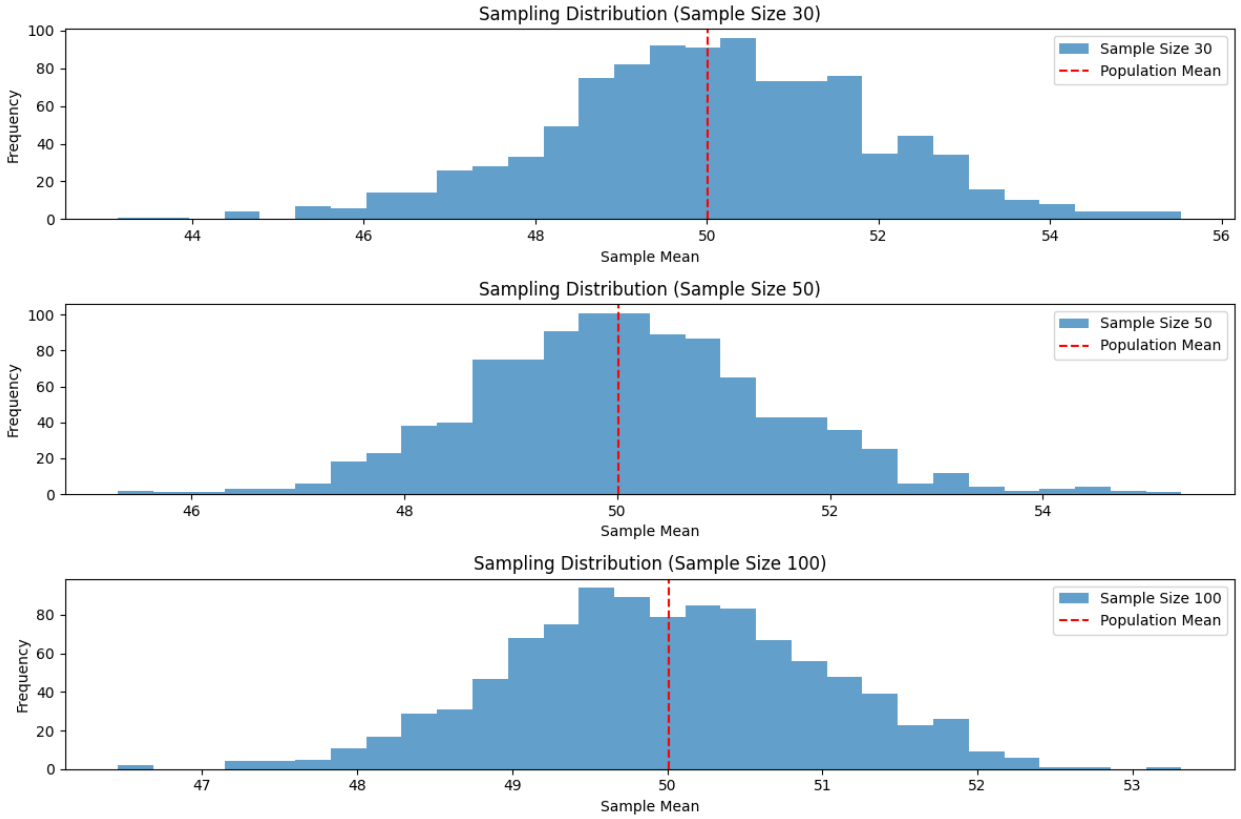
sample_means = {}

for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size,
replace=False)
        sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample
Size {size}')
    plt.axvline(np.mean(population), color='red', linestyle='dashed',
linewidth=1.5, label='Population Mean')
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```


Sampling Distribution (Sample Size 30)
Frequency
Sample Mean
Sample Size 30
Population Mean
Sampling Distribution (Sample Size 50)
Frequency
Sample Mean
Sample Size 50
Population Mean
Sampling Distribution (Sample Size 100)
Frequency
Sample Mean
Sample Size 100
Population Mean

Experiment No. 8

Conduct Z test for the data given using Python.

In [1]:
```
import numpy as np
from scipy import stats

marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])

mu_0 = 70

t_stat, p_value = stats.ttest_1samp(marks, mu_0)

print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from
70.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis → No significant difference.
```

In [2]:
```
from math import sqrt
from scipy.stats import norm

x_bar = 51.2
mu_0 = 50
sigma = 3
n = 36

z_stat = (x_bar - mu_0) / (sigma / sqrt(n))

p_value = 2 * (1 - norm.cdf(abs(z_stat)))

print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 50
g.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.
```

In [3]:
```
from scipy import stats

A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]

f_stat, p_value = stats.f_oneway(A, B, C)

print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05

if p_value < alpha:
    print("Reject Null Hypothesis → Means are significantly different.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
F-statistic: 25.923
P-value: 0.0011
Reject Null Hypothesis → Means are significantly different.
```

Experiment No. 9

Experiment to understand Matplotlib library use cases in Data Science through visualization.

Description: Use the Iris data set to understand the Matplotlib library.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
data=pd.read_csv('/content/Iris (1) - Iris (1).csv')
data
```


```
{"summary":"{\n  \"name\": \"data\",\n  \"rows\": 150,\n  \"fields\":
[\n    {\n      \"column\": \"sepal.length\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.8280661279778629,\n        \"min\": 4.3,\n        \"max\": 7.9,\n
\"num_unique_values\": 35,\n        \"samples\": [\n          6.2,\n
4.5,\n          5.6\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal.width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.435866284936698,\n        \"min\":
2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n
\"samples\": [\n          2.3,\n          4.0,\n          3.5\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"petal.length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.7652982332594667,\n        \"min\": 1.0,\n        \"max\": 6.9,\n
\"num_unique_values\": 43,\n        \"samples\": [\n          6.7,\n
3.8,\n          3.7\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal.width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.7622376689603465,\n        \"min\":
0.1,\n        \"max\": 2.5,\n        \"num_unique_values\": 22,\n
\"samples\": [\n          0.2,\n          1.2,\n          1.3\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"variety\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Setosa\",\n          \"Versicolor\",\n          \"Virginica\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"data"}
```


```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
```

```
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

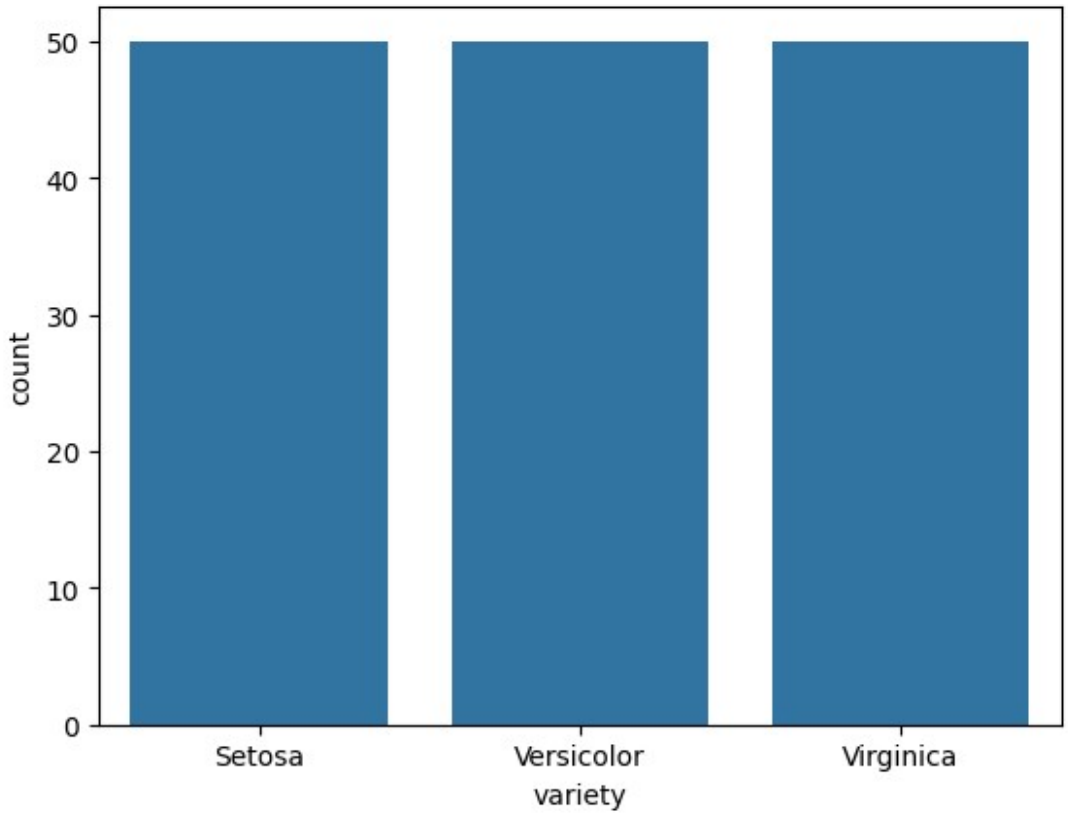
```
data.describe()
```


```
{"summary":"{\n  \"name\": \"data\",\n  \"rows\": 8,\n  \"fields\": [\
n    {\n      \"column\": \"sepal.length\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 51.24711349471842,\n
\"min\": 0.8280661279778629,\n        \"max\": 150.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
5.843333333333334,\n          5.8,\n          150.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sepal.width\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
52.08617800869866,\n        \"min\": 0.435866284936698,\n
\"max\": 150.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          3.057333333333337,\n          3.0,\n
150.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal.length\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 51.83521261418364,\n        \"min\":
1.0,\n        \"max\": 150.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          3.7580000000000005,\n          4.35,\n
150.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal.width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 52.636648242617504,\n        \"min\":
0.1,\n        \"max\": 150.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          1.1993333333333336,\n          1.3,\n
150.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
data.value_counts('variety')
```

```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

```
sns.countplot(x='variety',data=data,)
plt.show()
```

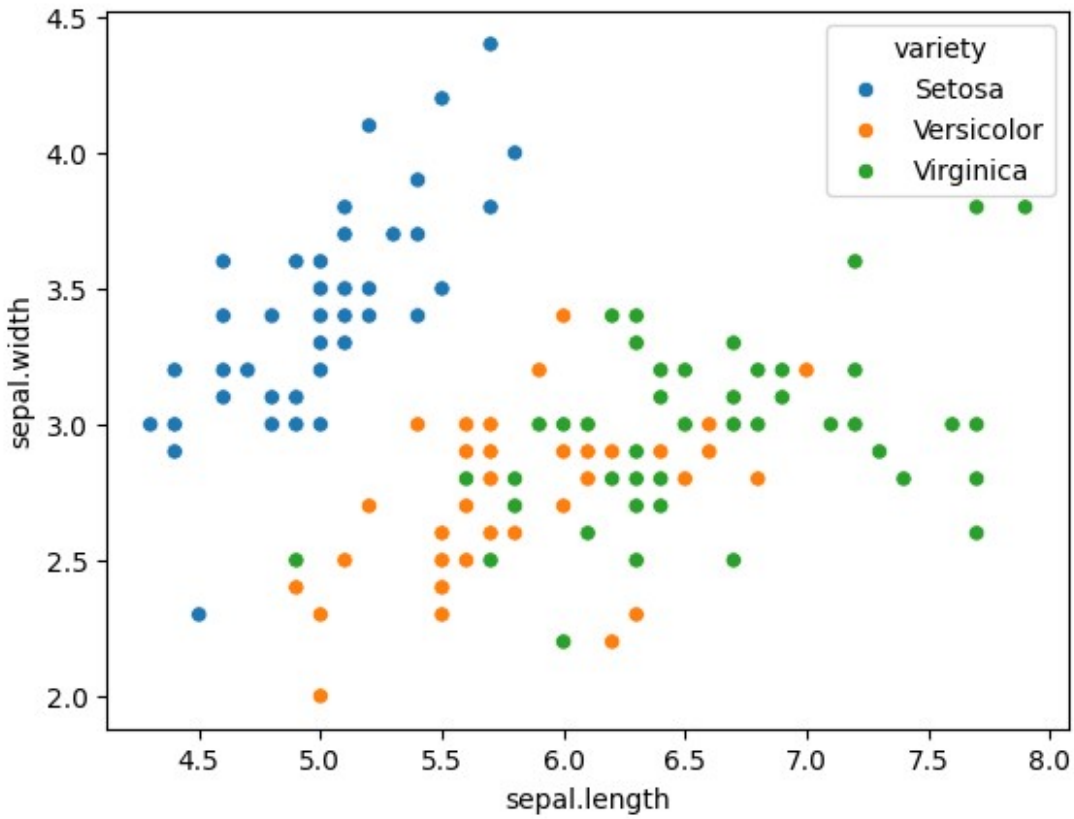



```
dummies=pd.get_dummies(data.variety)
FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,
[0,1,2,3]]],axis=1)
FinalDataset.head()
```

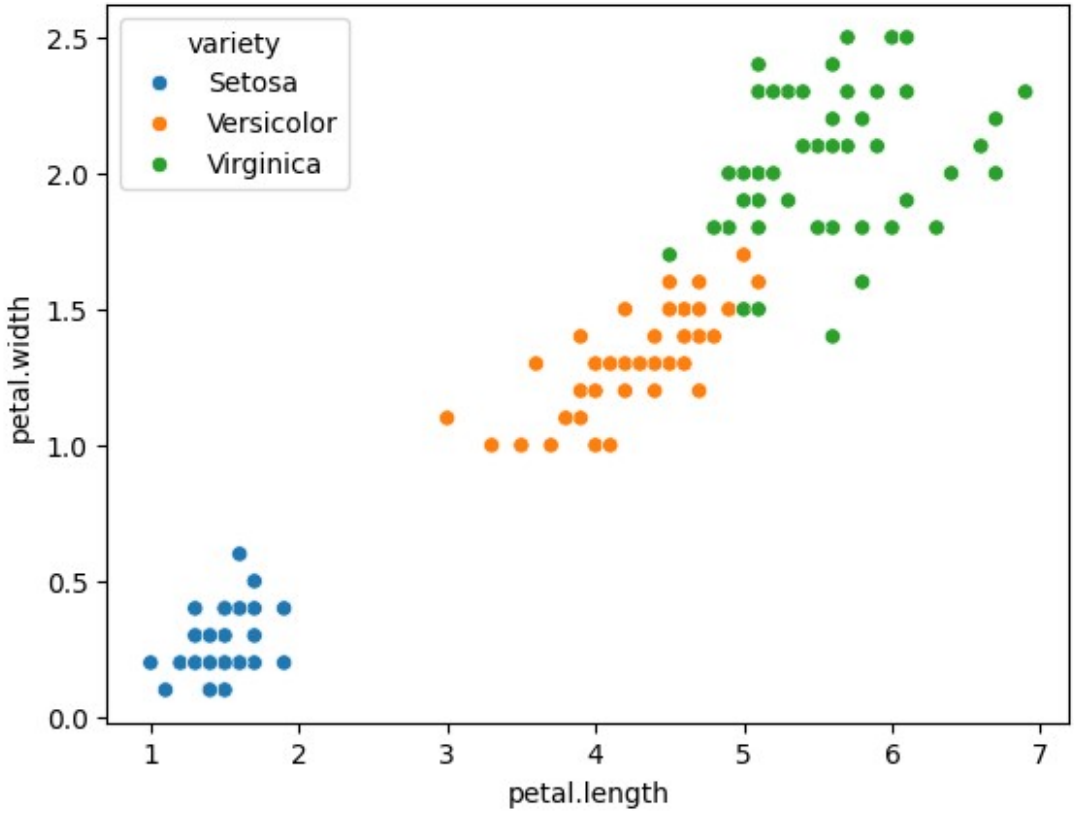

```
{"summary":"{\n  \"name\": \"FinalDataset\",\n  \"rows\": 150,\n
\"fields\": [\n    {\n      \"column\": \"Setosa\",\n
\"properties\": {\n        \"dtype\": \"boolean\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          false,\n
true\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Versicolor\",\n      \"properties\": {\n        \"dtype\":
\"boolean\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          true,\n          false\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Virginica\",\n
\"properties\": {\n        \"dtype\": \"boolean\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          true,\n
false\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal.length\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.8280661279778629,\n        \"min\":
4.3,\n        \"max\": 7.9,\n        \"num_unique_values\": 35,\n
\"samples\": [\n          6.2,\n          4.5\n        ],\n
```

```
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sepal.width\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.435866284936698,\n        \"min\": 2.0,\n        \"max\": 4.4,\n
\"num_unique_values\": 23,\n        \"samples\": [\n          2.3,\n
4.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal.length\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 1.7652982332594667,\n        \"min\":
1.0,\n        \"max\": 6.9,\n        \"num_unique_values\": 43,\n
\"samples\": [\n          6.7,\n          3.8\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"petal.width\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.7622376689603465,\n        \"min\": 0.1,\n        \"max\": 2.5,\n
\"num_unique_values\": 22,\n        \"samples\": [\n          0.2,\n
1.2\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"FinalDataset"}
```

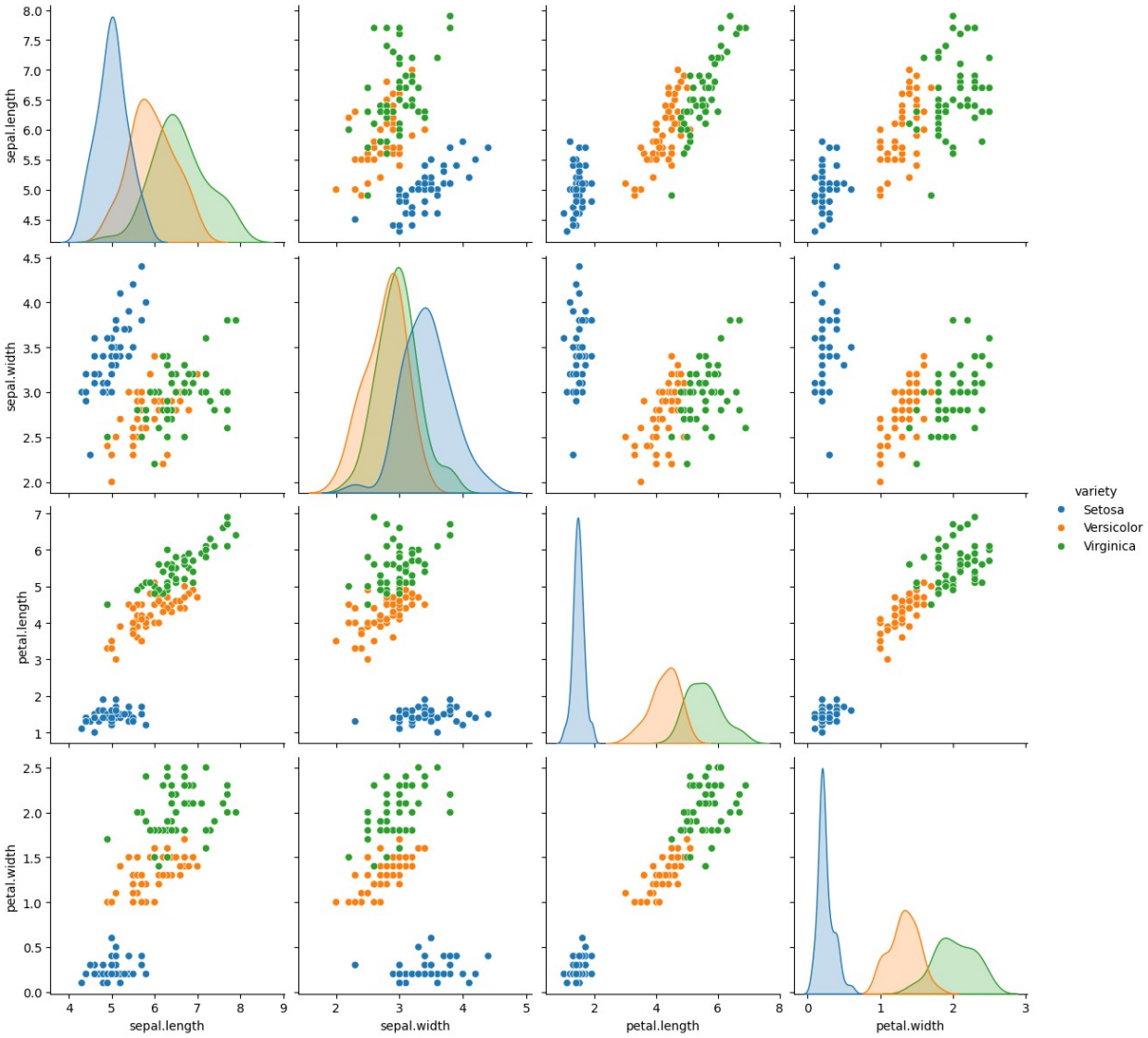

```
sns.scatterplot(x='sepal.length', y='sepal.width', hue='variety',
data=data)
plt.show()
```



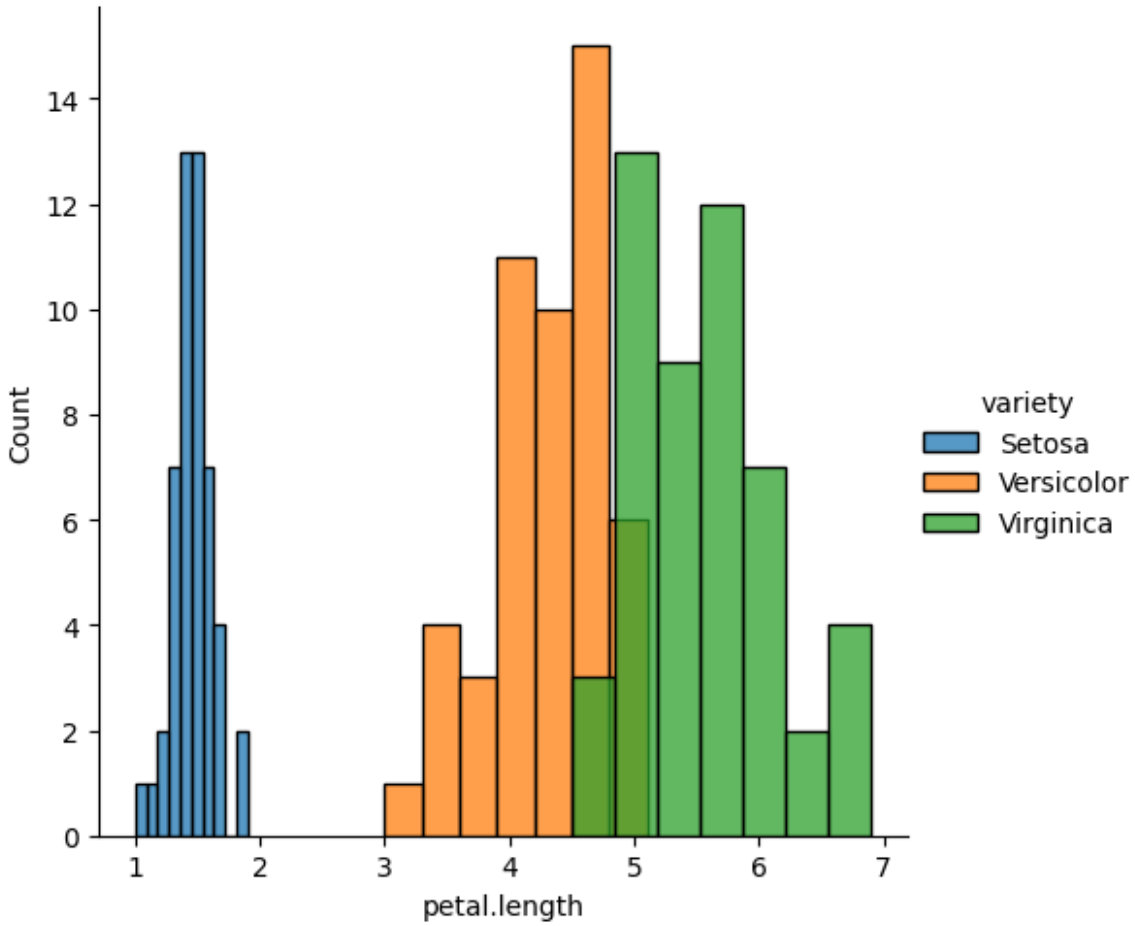
```
sns.scatterplot(x='petal.length', y='petal.width', hue='variety',
data=data)
plt.show()
```



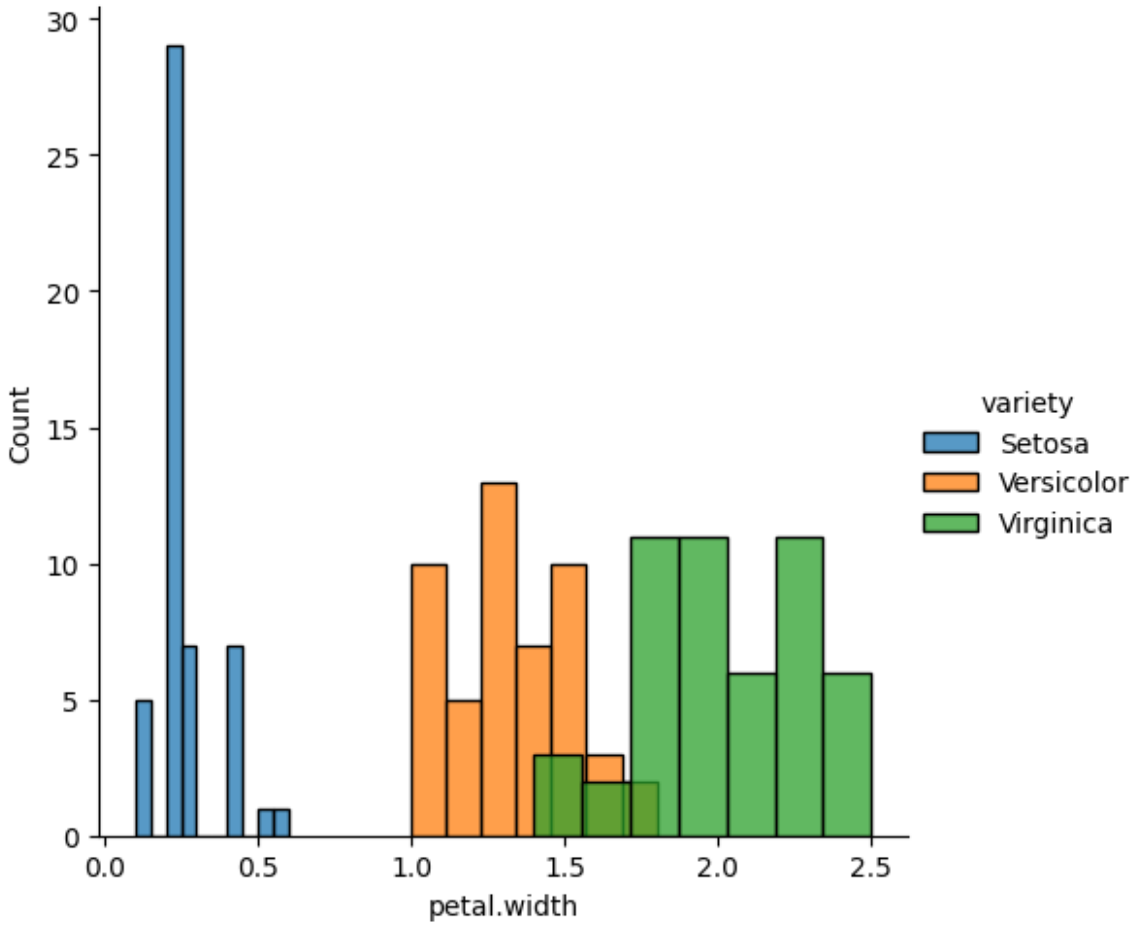

```
sns.pairplot(data,hue='variety',height=3);
```



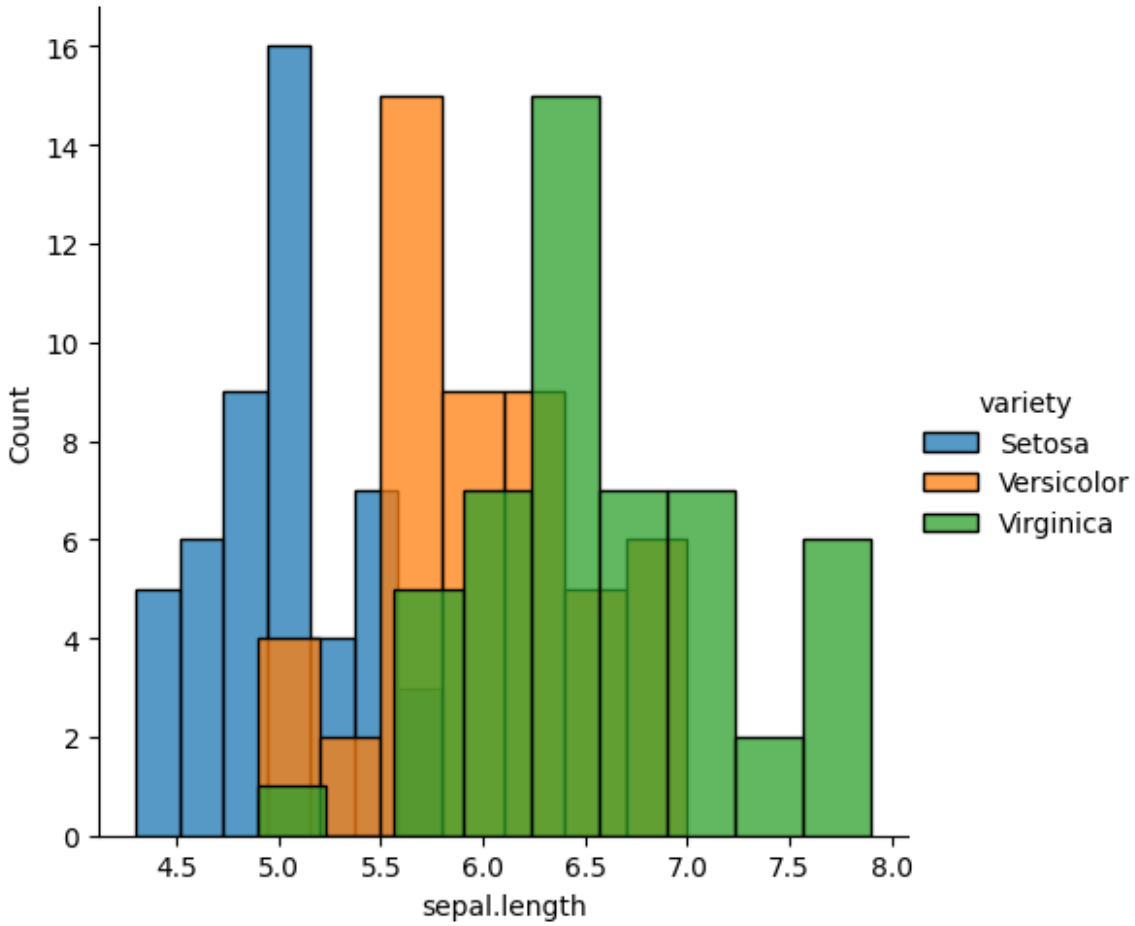

```
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'petal.length').add_legend();
plt.show();
```



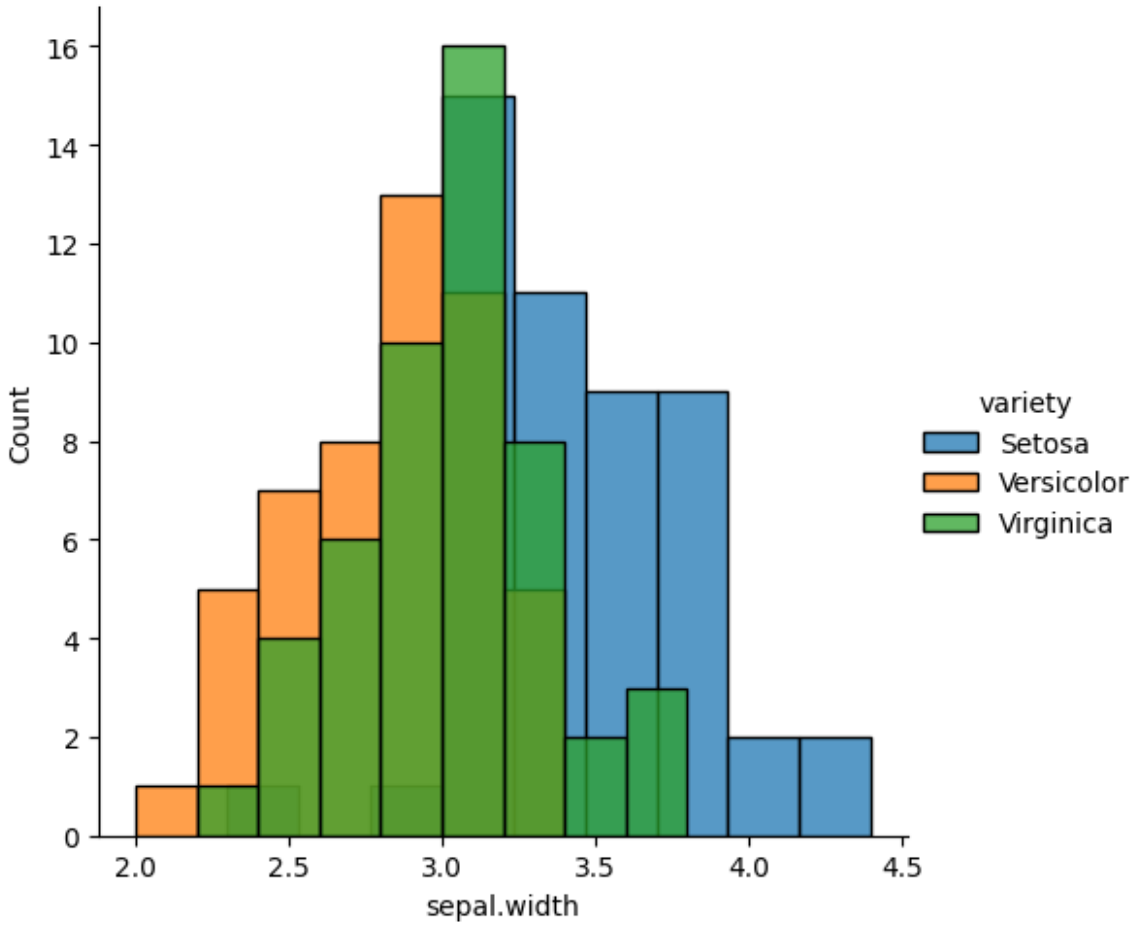

```
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'petal.width').add_legend();
plt.show();
```




```
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'sepal.length').add_legend();
plt.show();
```




```
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'sepal.width').add_legend();
plt.show();
```


Count
sepal.width
variety
Setosa
Versicolor
Virginica
0
2
4
6
8
10
12
14
16
2.0
2.5
3.0
3.5
4.0
4.5

Experiment No. 10

Experiment to understand array functions in Data Science.

Description: Understand array functions using the NumPy library.

```
import numpy as np
array=np.random.randint(1,100,9)
array
```

```
array([50, 69, 97,  1, 13, 52, 99, 62, 26])
```

```
np.sqrt(array)
```

```
array([7.07106781, 8.30662386, 9.8488578 , 1.        , 3.60555128,
       7.21110255, 9.94987437, 7.87400787, 5.09901951])
```

```
array.ndim
```

```
1
```

```
new_array=array.reshape(3,3)
new_array
```

```
array([[50, 69, 97],
       [ 1, 13, 52],
       [99, 62, 26]])
```

```
new_array.ndim
```

```
2
```

```
new_array.ravel()
```

```
array([50, 69, 97,  1, 13, 52, 99, 62, 26])
```

```
newm=new_array.reshape(3,3)
newm
```

```
array([[50, 69, 97],
       [ 1, 13, 52],
       [99, 62, 26]])
```

```
newm[2,1:3]
```

```
array([62, 26])
```

```
newm[1:2,1:3]
```

```
array([[13, 52]])
```

```
new_array[0:3,0:0]
```

```
array([], shape=(3, 0), dtype=int64)
```

```
new_array[0:2,0:1]
```

```
array([[50],
       [ 1]])
```

```
new_array[0:3,0:1]
```

```
array([[50],
       [ 1],
       [99]])
```

```
new_array[1:3]
```

```
array([[ 1, 13, 52],
       [99, 62, 26]])
```

Experiment No. 11

Experiment to understand pandas library use cases in Data Science.

Description: Understand DataFrame use cases using the pandas library.

```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
df
```

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 2,\n  \"fields\": [\n
{\n      \"column\": 0,\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 1,\n
\"max\": 2,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          2,\n          1\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": 1,\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          \"Jones\",\n          \"Smith\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": 2,\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 7071,\n        \"min\":
50000,\n        \"max\": 60000,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          60000,\n          50000\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```

```
df.columns=['Empd','Name','Salary']
df
```

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 2,\n  \"fields\": [\n
{\n      \"column\": \"Empd\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0,\n        \"min\": 1,\n
\"max\": 2,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          2,\n          1\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Name\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          \"Jones\",\n          \"Smith\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Salary\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 7071,\n
\"min\": 50000,\n        \"max\": 60000,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          60000,\n
50000\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
```

```
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 180.0+ bytes
```

```
df=pd.read_csv("/content/50_Startups.csv")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

```
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 50,\n  \"fields\": [\n
{\n      \"column\": \"R&D Spend\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 45902.25648230753,\n
\"min\": 0.0,\n        \"max\": 165349.2,\n
\"num_unique_values\": 49,\n        \"samples\": [\n
91992.39,\n          1000.23,\n          0.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Administration\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
28017.802755488683,\n        \"min\": 51283.14,\n        \"max\":
182645.56,\n        \"num_unique_values\": 50,\n        \"samples\":
[\n          135495.07,\n          82982.09,\n          115641.28\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Marketing Spend\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
122290.31072584528,\n        \"min\": 0.0,\n        \"max\":
471784.1,\n        \"num_unique_values\": 48,\n        \"samples\": [\
n          353183.81,\n          172795.67,\n          134050.07\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"State\",\n      \"properties\":
{\n        \"dtype\": \"category\",\n        \"num_unique_values\":
3,\n        \"samples\": [\n          \"New York\",\n
\"California\",\n          \"Florida\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Profit\",\n      \"properties\":
```

{\n   \"dtype\": \"number\",\n   \"std\":
40306.18033765055,\n   \"min\": 14681.4,\n   \"max\":
192261.83,\n   \"num_unique_values\": 50,\n   \"samples\":
[\n   134307.35,\n   81005.76,\n   99937.59\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
df.tail()
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 5,\n \"fields\": [\n
{\n   \"column\": \"R&D Spend\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 589.7833681700425,\n
\"min\": 0.0,\n   \"max\": 1315.46,\n
\"num_unique_values\": 4,\n   \"samples\": [\n   1315.46,\
n   542.05,\n   1000.23\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Administration\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
32849.625535718515,\n   \"min\": 51743.15,\n   \"max\":
135426.92,\n   \"num_unique_values\": 5,\n   \"samples\": [\
n   115816.21,\n   116983.8,\n   135426.92\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"Marketing Spend\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
129061.69240878528,\n   \"min\": 0.0,\n   \"max\":
297114.46,\n   \"num_unique_values\": 4,\n   \"samples\": [\
n   297114.46,\n   45173.06,\n   1903.93\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"State\",\n   \"properties\":
{\n   \"dtype\": \"string\",\n   \"num_unique_values\": 3,\n
\"samples\": [\n   \"New York\",\n   \"Florida\",\n
\"California\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Profit\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 18486.05628017047,\n   \"min\": 14681.4,\n
\"max\": 64926.08,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   49490.75,\n   14681.4,\n
42559.73\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
import numpy as np
import pandas as pd
df=pd.read_csv("/content/employee.csv")
df.head()
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 400,\n \"fields\": [\
n   {\n   \"column\": \"emp id\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 115,\n   \"min\": 1,\n
\"max\": 400,\n   \"num_unique_values\": 400,\n
\"samples\": [\n   210,\n   281,\n   34\

```
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 300,\n        \"samples\": [\n          \"KAVYA
M\",\n          \"KAVYA S\",\n          \"SRI B\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"salary\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 1811,\n
\"min\": 3000,\n        \"max\": 10000,\n
\"num_unique_values\": 8,\n        \"samples\": [\n          6000,\n
4000,\n          5000\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


```
df.tail()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 5,\n  \"fields\": [\n
{\n      \"column\": \"emp id\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 396,\n
\"max\": 400,\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          397,\n          400,\n          398\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"name\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \"TANVI V\",\n          \"YASH C\",\n
\"UMESH A\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"salary\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 2588,\n        \"min\": 3000,\n        \"max\": 10000,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          10000,\n
7000,\n          5000\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  400 non-null    int64
 1   name    400 non-null    object
 2   salary  400 non-null    int64
dtypes: int64(2), object(1)
memory usage: 9.5+ KB
```

```
df.salary
```

```
0       5000
1       6000
2       7000
```

```
3       5000
4       8000
       ...
395     3000
396    10000
397     5000
398     6000
399     7000
Name: salary, Length: 400, dtype: int64
```

```
type(df.salary)
```

```
pandas.core.series.Series
```

```
df.salary.mean()
```

```
np.float64(6005.0)
```

```
df.salary.median()
```

```
6000.0
```

```
df.salary.mode()
```

```
0    5000
1    6000
Name: salary, dtype: int64
```

```
df.salary.var()
```

```
3283182.9573934837
```

```
df.salary.std()
```

```
1811.9555616497562
```

```
df.describe()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n{\n      \"column\": \"emp id\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 143.85727156983504,\n\"min\": 1.0,\n        \"max\": 400.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          400.0,\n          200.5,\n300.25\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"salary\",\n      \"properties\": {\n        \"dtype\": \"number\",\n\"std\": 3076.5706213445333,\n        \"min\": 400.0,\n\"max\": 10000.0,\n        \"num_unique_values\": 8,\n\"samples\": [\n          6005.0,\n          6000.0,\n          400.0\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
df.describe(include='all')
```

{"summary":"{\n \"name\": \"df\",\n \"rows\": 11,\n \"fields\": [\n{\n   \"column\": \"emp id\",\n   \"properties\": {\n\"dtype\": \"number\",\n   \"std\": 143.85727156983504,\n\"min\": 1.0,\n   \"max\": 400.0,\n   \"num_unique_values\": 6,\n   \"samples\": [\n   400.0,\n   200.5,\n300.25\n   ],\n   \"semantic_type\": \"\",\n\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"name\",\n   \"properties\": {\n   \"dtype\": \"category\",\n\"num_unique_values\": 4,\n   \"samples\": [\n   300,\n\"3\",\n   \"400\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n\"column\": \"salary\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 3076.5706213445333,\n   \"min\": 400.0,\n   \"max\": 10000.0,\n   \"num_unique_values\": 8,\n\"samples\": [\n   6005.0,\n   6000.0,\n   400.0\n   ],\n   \"semantic_type\": \"\",\n\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
empCol=df.columns
empCol
```

```
Index(['emp id', 'name', 'salary'], dtype='object')
```

```
emparray=df.values
emparray
```

```
array([[1, 'ANANYA R', 5000],
       [2, 'BHAVESH K', 6000],
       [3, 'SREYA G', 7000],
       ...,
       [398, 'UMESH A', 5000],
       [399, 'VARUN B', 6000],
       [400, 'YASH C', 7000]], dtype=object)
```

```
employee_DF=pd.DataFrame(emparray,columns=empCol)
employee_DF
```

{"summary":"{\n \"name\": \"employee_DF\",\n \"rows\": 400,\n\"fields\": [\n   {\n   \"column\": \"emp id\",\n\"properties\": {\n   \"dtype\": \"date\",\n   \"min\": 1,\n\"max\": 400,\n   \"num_unique_values\": 400,\n\"samples\": [\n   210,\n   281,\n   34\n   ],\n   \"semantic_type\": \"\",\n\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"name\",\n   \"properties\": {\n   \"dtype\": \"string\",\n\"num_unique_values\": 300,\n   \"samples\": [\n   \"KAVYA M\",\n   \"KAVYA S\",\n   \"SRI B\"\n   ],\n\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"salary\",\n   \"properties\": {\n   \"dtype\": \"date\",\n   \"min\": 3000,\n\"max\": 10000,\n   \"num_unique_values\": 8,\n

```
\"samples\": [\n          6000,\n          4000,\n          5000\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"employee_DF"}
```

Experiment No. 12

Experiment to detect outliers in a given data set.

```
import numpy as np
import pandas as pd
df=pd.read_csv('outlier_data.csv')
print(df.head())
print(df.describe())


salary=df['Salary'].values
print(np.percentile(salary,25))
print(np.percentile(salary,50))
print(np.percentile(salary,75))


def outDetection(array):
    Q1,Q3=np.percentile(array,[25,75])
    IQR=Q3-Q1
    lr=Q1-(1.5*IQR)
    ur=Q3+(1.5*IQR)
    return lr,ur

lr,ur=outDetection(salary)
print("lower range =",lr,"upper range =",ur)

import seaborn as sns
import matplotlib.pyplot as plt
sns.boxplot(x=df['Salary'])
plt.show()

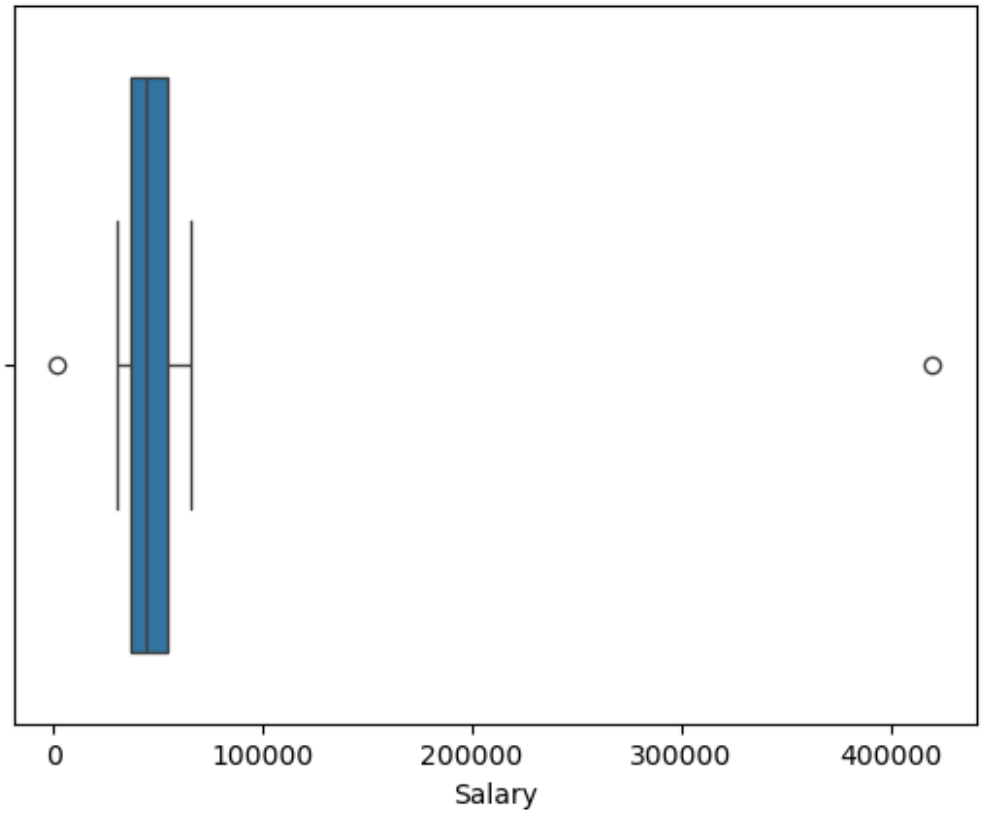

print(df[(df.Salary<lr)|(df.Salary>ur)])

new_df=df[(df.Salary>lr)&(df.Salary<ur)]
print(new_df)

lr1,ur1=outDetection(new_df.Salary.values)
print(lr1,ur1)
final_df=new_df[(new_df.Salary>lr1)&(new_df.Salary<ur1)]
print(final_df)
sns.boxplot(x=final_df['Salary'])
plt.show()
```

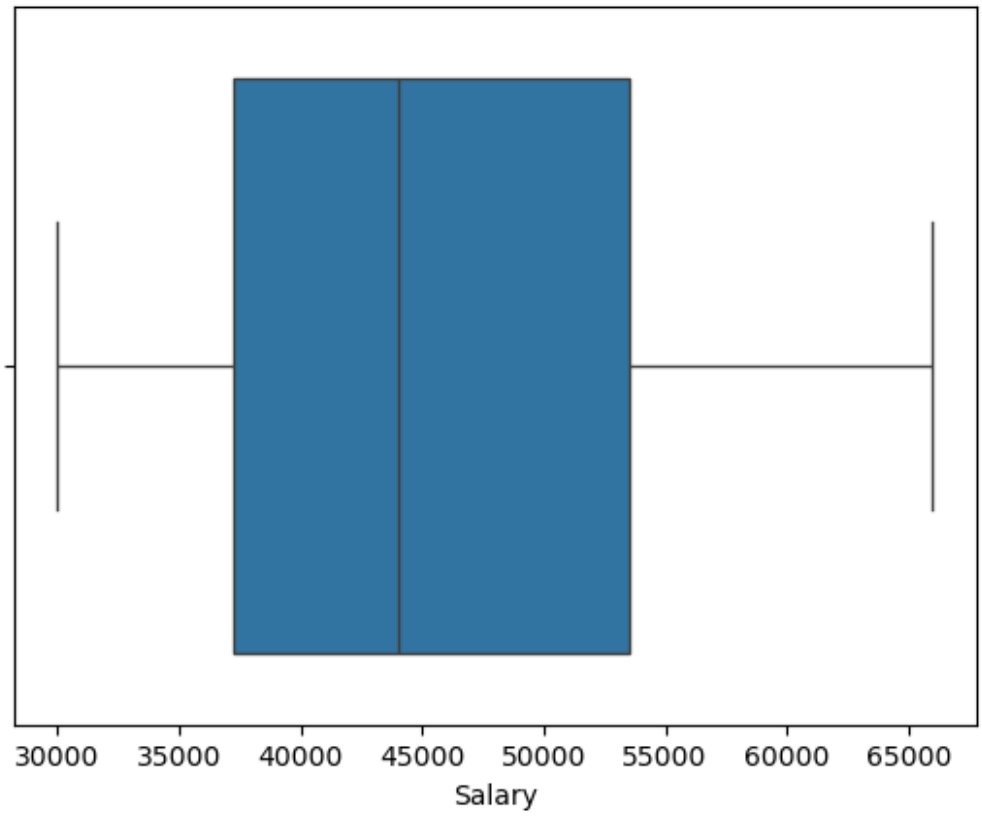
```
  Employee  Age  Experience  Salary
0      E01   25           2   32000
1      E02   28           4   38000
2      E03   30           6   42000
3      E04   27           3   35000
4      E05   35          10   55000
             Age  Experience         Salary
count  24.000000   24.000000      24.000000
mean   32.000000    6.541667   59333.333333
```

```
std     6.303898    3.956054   77956.323706
min    24.000000    1.000000    2000.000000
25%    27.750000    3.000000   36750.000000
50%    30.500000    6.000000   44000.000000
75%    35.000000    9.250000   54250.000000
max    52.000000   15.000000  420000.000000
36750.0
44000.0
54250.0
lower range = 10500.0 upper range = 80500.0
```




```
   Employee  Age  Experience  Salary
19      E20   52           3  420000
22      E23   26           2    2000
   Employee  Age  Experience  Salary
0       E01   25           2   32000
1       E02   28           4   38000
2       E03   30           6   42000
3       E04   27           3   35000
4       E05   35          10   55000
5       E06   32           8   48000
6       E07   29           5   40000
7       E08   41          15   66000
```

```
8        E09   38           12   58000
9        E10   26            2   33000
10       E11   34            9   52000
11       E12   31            7   45000
12       E13   36           11   57000
13       E14   24            1   30000
14       E15   33            8   50000
15       E16   29            5   41000
16       E17   40           14   63000
17       E18   27            3   36000
18       E19   30            6   43000
20       E21   28            4   37000
21       E22   35           10   54000
23       E24   32            7   47000
12875.0 77875.0
   Employee  Age  Experience  Salary
0        E01   25            2   32000
1        E02   28            4   38000
2        E03   30            6   42000
3        E04   27            3   35000
4        E05   35           10   55000
5        E06   32            8   48000
6        E07   29            5   40000
7        E08   41           15   66000
8        E09   38           12   58000
9        E10   26            2   33000
10       E11   34            9   52000
11       E12   31            7   45000
12       E13   36           11   57000
13       E14   24            1   30000
14       E15   33            8   50000
15       E16   29            5   41000
16       E17   40           14   63000
17       E18   27            3   36000
18       E19   30            6   43000
20       E21   28            4   37000
21       E22   35           10   54000
23       E24   32            7   47000
```


30000
35000
40000
45000
50000
55000
60000
65000
Salary

Experiment No. 13

Experiment to handle missing values and inappropriate data in a given data set.

```
import numpy as np
import pandas as pd

data = {
    "Customer ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Age Group": [
        "20-25", "30-35", "25-30", "20-25", "35+", "35+",
        "25+", "20-25", "25-30", "30-35"
    ],
    "Rating(1-5)": [4, 5, 6, -1, 3, 3, 4, 7, 2, 5],
    "Hotel": [
        "Taj", "Taj", "Lemon Tree", "Red Fox", "Red Fox",
        "Taj", "Taj", "Lemon Tree", "Lemon Tree", "Taj"
    ],
    "Food preference": [
        "Veg", "Non-veg", "Veg", "Non-veg", "Veg",
        "Non-veg", "Vegetarian", "Non-veg", "Veg", "Vegetarian"
    ],
    "Bill": [
        300, 200, 1322, 1234, 989, 1905, 1000, 2999, 3456, 3456
    ],
    "No of pax": [2, 3, 2, 2, 2, 1, 1, 3, 3, 5],
    "Estimated salary": [
        45000, 50000, 120000, 45000, 122220, 211234,
        345623, 30000, 99999, 99999
    ]
}

df = pd.DataFrame(data)

df = df.drop_duplicates()

df["Hotel"] = df["Hotel"].replace(["Taj"], "Taj")

df["Food preference"] = df["Food preference"].replace(
    {"Vegetarian": "Veg"}
)

df["Rating(1-5)"] = pd.to_numeric(
    df["Rating(1-5)"], errors="coerce"
)

df.loc[
    (df["Rating(1-5)"] < 1) | (df["Rating(1-5)"] > 5),
    "Rating(1-5)"
] = np.nan

df["Bill"] = pd.to_numeric(df["Bill"], errors="coerce")
df.loc[df["Bill"] <= 0, "Bill"] = np.nan
```

```
df["Estimated salary"] = pd.to_numeric(
    df["Estimated salary"], errors="coerce"
)

df.loc[df["Estimated salary"] <= 0, "Estimated salary"] = np.nan

df["Age Group"] = df["Age Group"].fillna(
    df["Age Group"].mode()[0]
)

df["Hotel"] = df["Hotel"].fillna(df["Hotel"].mode()[0])

df["Food preference"] = df["Food preference"].fillna(
    df["Food preference"].mode()[0]
)

df["Rating(1-5)"] = df["Rating(1-5)"].fillna(
    df["Rating(1-5)"].median()
)

df["No of pax"] = pd.to_numeric(df["No of pax"], errors="coerce")
df["No of pax"] = df["No of pax"].fillna(
    df["No of pax"].median()
)

df["Bill"] = df["Bill"].fillna(df["Bill"].mean())

df["Estimated salary"] = df["Estimated salary"].fillna(
    df["Estimated salary"].median()
)

print(df)
```

```
   Customer ID Age Group  Rating(1-5)       Hotel Food preference
Bill  \
0            1     20-25          4.0         Taj             Veg
300.0
1            2     30-35          5.0         Taj         Non-veg
200.0
2            3     25-30          4.0  Lemon Tree             Veg
1322.0
3            4     20-25          4.0     Red Fox         Non-veg
1234.0
4            5       35+          3.0     Red Fox             Veg
989.0
5            6       35+          3.0         Taj         Non-veg
1905.0
6            7       25+          4.0         Taj             Veg
1000.0
7            8     20-25          4.0  Lemon Tree         Non-veg
```

```
2999.0
8            9     25-30          2.0  Lemon Tree              Veg
3456.0
9           10     30-35          5.0         Taj              Veg
3456.0

   No of pax  Estimated salary
0          2           45000.0
1          3           50000.0
2          2          120000.0
3          2           45000.0
4          2          122220.0
5          1          211234.0
6          1          345623.0
7          3           30000.0
8          3           99999.0
9          5           99999.0
```

Experiment No. 14

Experiment to understand feature scaling.

```python
import numpy as np
import pandas as pd

data = {
    "Country": [
        "France", "Spain", "Germany", "Spain", "Germany",
        "Spain", "Germany", "France", "Spain", "France",
        "Germany", "France"
    ],
    "Age": [42, 27, 30, 38, 40, 75, np.nan, 48, 50, 37, 29, 35],
    "Salary": [
        72000, 48000, 54000, 61000, np.nan, 58000,
        52000, 79000, 83000, 67000, 65000, 59000
    ],
    "Purchased": [
        "No", "Yes", "No", "No", "Yes", "Yes",
        "Yes", "No", "Yes", "No", "Yes", "No"
    ]
}

df = pd.DataFrame(data)

df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())

df = pd.get_dummies(df, columns=["Country"], dtype=int)

df["Purchased"] = np.where(df["Purchased"] == "Yes", 1, 0)

df["Age"] = (
    df["Age"] - np.mean(df["Age"])
) / np.std(df["Age"])

df["Salary"] = (
    df["Salary"] - np.mean(df["Salary"])
) / np.std(df["Salary"])

print(df)
```

```
        Age        Salary  Purchased  Country_France  Country_Germany
\
0   0.081424  8.451850e-01          0               1                0

1  -1.139933 -1.528526e+00          1               0                0

2  -0.895662 -9.350983e-01          0               0                1

3  -0.244271 -2.427659e-01          0               0                0

4  -0.081424  7.196259e-16          1               0                1
```

```
5   2.768409 -5.394798e-01           1                0                0
6   0.000000 -1.132908e+00           1                0                1
7   0.569967  1.537517e+00           0                1                0
8   0.732814  1.933136e+00           1                0                0
9  -0.325695  3.506619e-01           0                1                0
10 -0.977086  1.528526e-01           1                0                1
11 -0.488543 -4.405752e-01           0                1                0
```

```
    Country_Spain
0               0
1               1
2               0
3               1
4               0
5               1
6               0
7               0
8               1
9               0
10              0
11              0
```

Experiment No. 15

Experiment to understand the data preprocessing in Data Science.

```
import numpy as np
import pandas as pd
data=pd.read_csv('scaling_data.csv')
print(data)
print("\n")

data.Branch=data.Branch.fillna(data.Branch.mode()[0])
X=data.iloc[:,:-1].values
y=data.iloc[:,-1].values


from sklearn.impute import SimpleImputer
cg=SimpleImputer(strategy="mean",missing_values=np.nan)
stip=SimpleImputer(strategy="mean",missing_values=np.nan)
X[:,[1]]=cg.fit_transform(X[:,[1]])
X[:,[2]]=stip.fit_transform(X[:,[2]])
print(X)
print("\n")

from sklearn.preprocessing import OneHotEncoder
ohe=OneHotEncoder(sparse_output=False)
Branch=ohe.fit_transform(X[:,[0]])
print(Branch)
print('\n')
ready=np.concatenate((Branch,X[:,[1,2]]),axis=1)
print(ready,end = "\n")


from sklearn.preprocessing import StandardScaler
ss=StandardScaler()
std_data=ss.fit_transform(ready)
print(std_data,end = "\n")


from sklearn.preprocessing import MinMaxScaler
mm=MinMaxScaler(feature_range=(0,1))
minmax_data=mm.fit_transform(ready)
print(minmax_data,end = '\n')
```

```
   Branch  CGPA  Stipend Placed
0     CSE   8.9  25000.0    Yes
1     ECE   7.2  12000.0     No
2    MECH   6.8   8000.0     No
3     CSE   9.3  30000.0    Yes
4      IT   8.1  18000.0    Yes
5     ECE   NaN  15000.0     No
6    MECH   7.5  10000.0    Yes
7     CSE   8.4      NaN    Yes
8      IT   6.9   9000.0     No
9     NaN   7.8  14000.0     No
```

```
10    ECE   8.6  20000.0    Yes
11     IT   7.1  11000.0     No
```

```
[['CSE' 8.9 25000.0]
 ['ECE' 7.2 12000.0]
 ['MECH' 6.8 8000.0]
 ['CSE' 9.3 30000.0]
 ['IT' 8.1 18000.0]
 ['ECE' 7.872727272727272 15000.0]
 ['MECH' 7.5 10000.0]
 ['CSE' 8.4 15636.363636363636]
 ['IT' 6.9 9000.0]
 ['CSE' 7.8 14000.0]
 ['ECE' 8.6 20000.0]
 ['IT' 7.1 11000.0]]
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 0. 1.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [0. 1. 0. 0.]
 [0. 0. 0. 1.]
 [1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]]
```

```
[[1.0 0.0 0.0 0.0 8.9 25000.0]
 [0.0 1.0 0.0 0.0 7.2 12000.0]
 [0.0 0.0 0.0 1.0 6.8 8000.0]
 [1.0 0.0 0.0 0.0 9.3 30000.0]
 [0.0 0.0 1.0 0.0 8.1 18000.0]
 [0.0 1.0 0.0 0.0 7.872727272727272 15000.0]
 [0.0 0.0 0.0 1.0 7.5 10000.0]
 [1.0 0.0 0.0 0.0 8.4 15636.363636363636]
 [0.0 0.0 1.0 0.0 6.9 9000.0]
 [1.0 0.0 0.0 0.0 7.8 14000.0]
 [0.0 1.0 0.0 0.0 8.6 20000.0]
 [0.0 0.0 1.0 0.0 7.1 11000.0]]
[[ 1.41421356e+00 -5.77350269e-01 -5.77350269e-01 -4.47213595e-01
   1.32236874e+00  1.46452077e+00]
 [-7.07106781e-01  1.73205081e+00 -5.77350269e-01 -4.47213595e-01
  -8.65975986e-01 -5.68745929e-01]
 [-7.07106781e-01 -5.77350269e-01 -5.77350269e-01  2.23606798e+00
  -1.38088063e+00 -1.19436645e+00]
```

```
 [ 1.41421356e+00 -5.77350269e-01 -5.77350269e-01 -4.47213595e-01
   1.83727338e+00  2.24654642e+00]
 [-7.07106781e-01 -5.77350269e-01  1.73205081e+00 -4.47213595e-01
   2.92559455e-01  3.69684854e-01]
 [-7.07106781e-01  1.73205081e+00 -5.77350269e-01 -4.47213595e-01
   0.00000000e+00 -9.95305376e-02]
 [-7.07106781e-01 -5.77350269e-01 -5.77350269e-01  2.23606798e+00
  -4.79797506e-01 -8.81556190e-01]
 [ 1.41421356e+00 -5.77350269e-01 -5.77350269e-01 -4.47213595e-01
   6.78737935e-01 -2.84499275e-16]
 [-7.07106781e-01 -5.77350269e-01  1.73205081e+00 -4.47213595e-01
  -1.25215447e+00 -1.03796132e+00]
 [ 1.41421356e+00 -5.77350269e-01 -5.77350269e-01 -4.47213595e-01
  -9.36190256e-02 -2.55935668e-01]
 [-7.07106781e-01  1.73205081e+00 -5.77350269e-01 -4.47213595e-01
   9.36190256e-01  6.82495115e-01]
 [-7.07106781e-01 -5.77350269e-01  1.73205081e+00 -4.47213595e-01
  -9.94702147e-01 -7.25151060e-01]]
[[1.         0.         0.         0.         0.84       0.77272727]
 [0.         1.         0.         0.         0.16       0.18181818]
 [0.         0.         0.         1.         0.         0.        ]
 [1.         0.         0.         0.         1.         1.        ]
 [0.         0.         1.         0.         0.52       0.45454545]
 [0.         1.         0.         0.         0.42909091 0.31818182]
 [0.         0.         0.         1.         0.28       0.09090909]
 [1.         0.         0.         0.         0.64       0.34710744]
 [0.         0.         1.         0.         0.04       0.04545455]
 [1.         0.         0.         0.         0.4        0.27272727]
 [0.         1.         0.         0.         0.72       0.54545455]
 [0.         0.         1.         0.         0.12       0.13636364]]
```

Experiment No. 16

Experiment to understand EDA – Quantitative and Qualitative analysis.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)

n = 2503

shopping = pd.DataFrame({
    "Purchase_Amount": np.random.randint(200, 5000, n),
    "Discount": np.random.randint(0, 50, n),
    "Page": np.random.randint(1, 20, n),
    "Gender": np.random.choice(["Male", "Female"], n),
    "Category": np.random.choice(
        ["Electronics", "Clothing", "Home"], n
    ),
    "Payment": np.random.choice(
        ["UPI", "Card", "Cash"], n
    ),
    "Delivery": np.random.choice(
        ["Standard", "Express"], n
    )
})

shopping["Final_Price"] = (
    shopping["Purchase_Amount"]
    - shopping["Purchase_Amount"] * shopping["Discount"] / 100
)

print(shopping.head())
print(shopping.info())
print(shopping.describe())

sns.jointplot(
    x="Discount",
    y="Final_Price",
    data=shopping,
    kind="hex"
)
plt.show()

x = "Page"
y = "Purchase_Amount"

sns.jointplot(
    x=x,
    y=y,
    data=shopping,
    kind="hex"
```

```
)
plt.show()

plt.figure(figsize=(6, 4))

sns.heatmap(
    shopping.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
plt.show()
```

```
   Purchase_Amount  Discount  Page  Gender  Category Payment  Delivery
\
0             1060        27     1    Male  Clothing     UPI   Express

1             3972        28     6  Female  Clothing     UPI  Standard

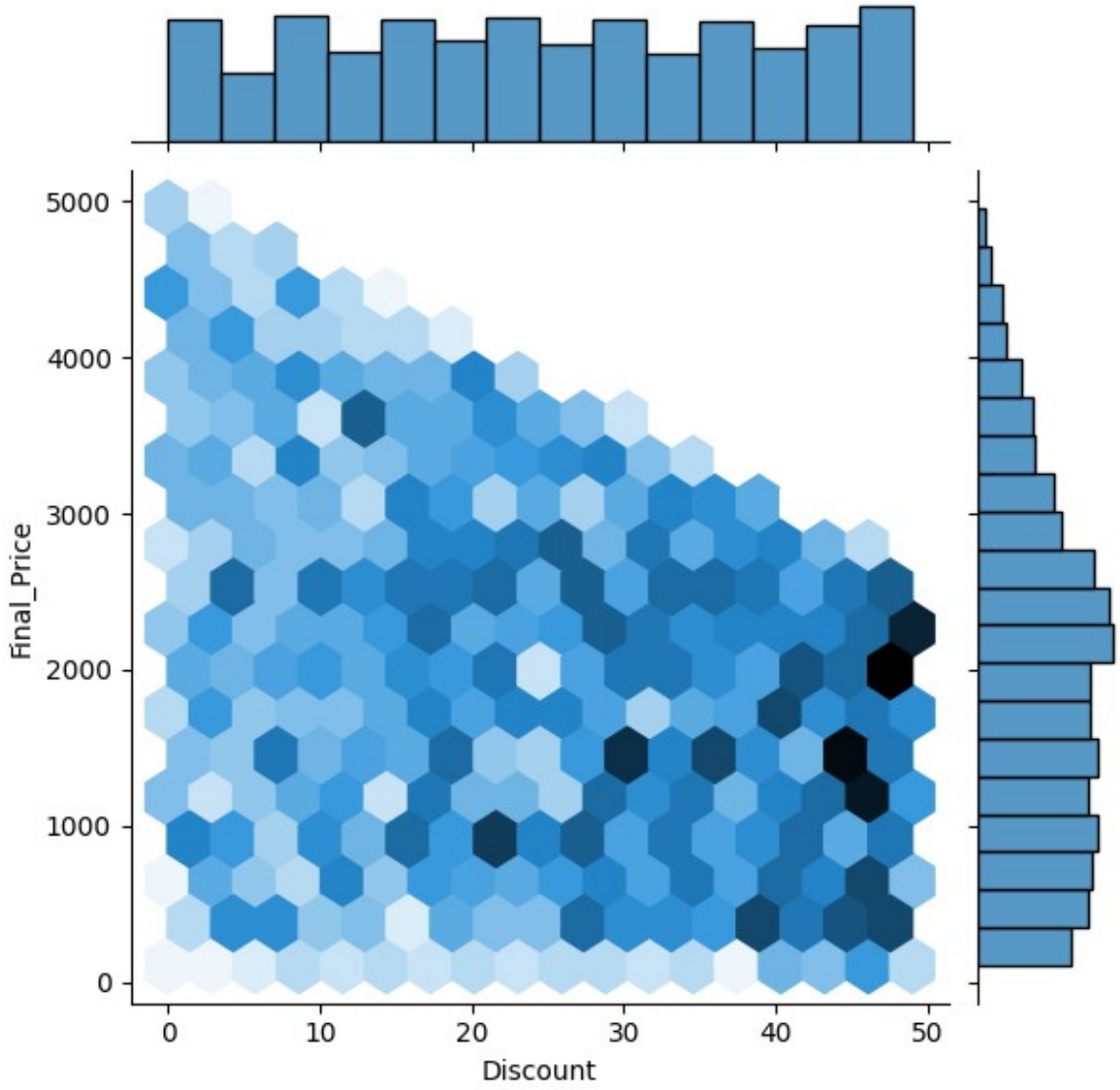
2             3292         8    15  Female  Clothing    Card   Express

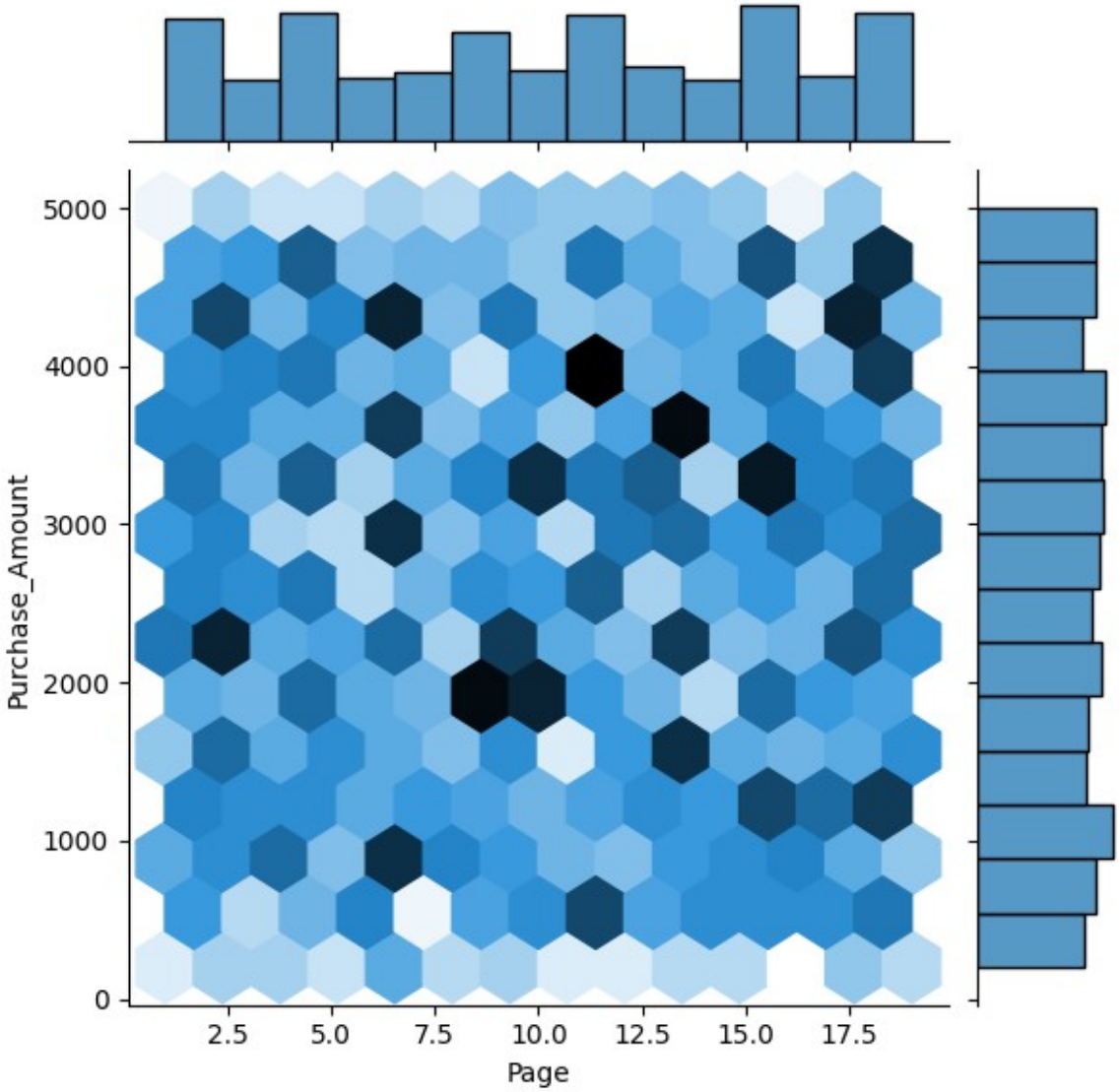
3              666        37    14    Male  Clothing    Card  Standard

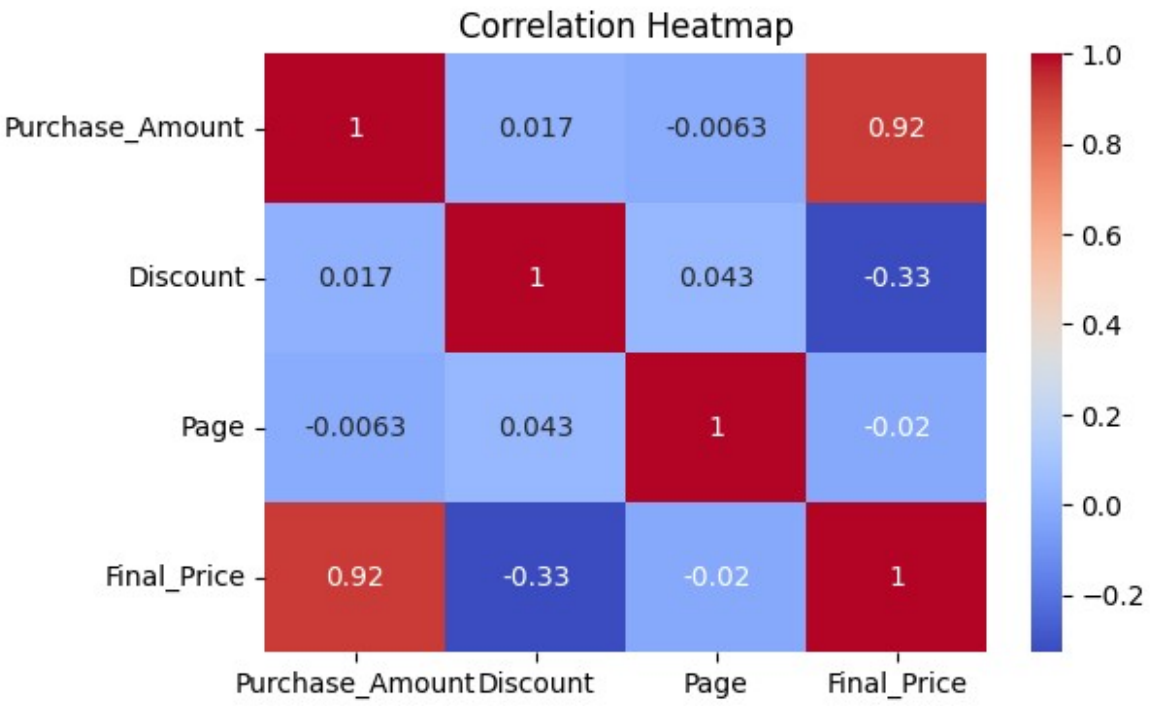
4             4626        49    15  Female      Home    Card   Express


   Final_Price
0       773.80
1      2859.84
2      3028.64
3       419.58
4      2359.26
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2503 entries, 0 to 2502
Data columns (total 8 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Purchase_Amount  2503 non-null   int64
 1   Discount         2503 non-null   int64
 2   Page             2503 non-null   int64
 3   Gender           2503 non-null   object
 4   Category         2503 non-null   object
 5   Payment          2503 non-null   object
 6   Delivery         2503 non-null   object
 7   Final_Price      2503 non-null   float64
dtypes: float64(1), int64(3), object(4)
memory usage: 156.6+ KB
None
       Purchase_Amount     Discount         Page  Final_Price
```

count	2503.000000	2503.000000	2503.000000	2503.000000
mean	2612.968038	24.866560	10.136636	1959.905513
std	1370.582778	14.362769	5.462366	1105.941082
min	201.000000	0.000000	1.000000	109.080000
25%	1443.000000	12.000000	5.000000	1031.490000
50%	2633.000000	25.000000	10.000000	1921.830000
75%	3778.000000	37.000000	15.000000	2714.135000
max	4999.000000	49.000000	19.000000	4952.000000




5000
4000
3000
2000
1000
0
Purchase_Amount
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Page


Correlation Heatmap
Purchase_Amount
Discount
Page
Final_Price
1
0.017
-0.0063
0.92
0.017
1
0.043
-0.33
-0.0063
0.043
1
-0.02
0.92
-0.33
-0.02
1
Purchase_Amount
Discount
Page
Final_Price
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

Experiment No. 17

Experiment to understand Linear Regression for a given data set.

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Create sample salary data
df=pd.DataFrame({
    "YearsExperience": [1.1, 1.5, 2, 2.2, 2.9, 3.2, 3.7, 4,
                        4.5, 4.9, 5.3, 5.9, 6.2, 6.8, 7.1,
                        7.9, 8.2, 8.7, 9, 9.5, 10, 10.5],
    "Salary": [39343, 37731, 43525, 39891, 56642, 54445, 57189,
               55794, 61111, 67938, 66029, 81363, 93940, 91738,
               98273, 101302, 113812, 109431, 105582, 116969,
               122391, 121872]
})
print(df.head())
print(df.info())
print(df.describe())
features = df[["YearsExperience"]].values
label = df[["Salary"]].values
x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(x_train, y_train)
print("Training Score:", model.score(x_train, y_train))
print("Testing Score:", model.score(x_test, y_test))
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
plt.scatter(features, label)
plt.plot(features, model.predict(features), color="red")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Linear Regression - Salary Prediction")
plt.show()
years = np.array([[5.0]])
salary = model.predict(years)
print("Estimated Salary for 5 years of experience:", salary[0][0])
```

```
   YearsExperience  Salary
0              1.1   39343
1              1.5   37731
2              2.0   43525
3              2.2   39891
4              2.9   56642
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22 entries, 0 to 21
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
```

```
---  ------           --------------  -----
 0   YearsExperience  22 non-null     float64
 1   Salary           22 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 484.0 bytes
None
       YearsExperience         Salary
count        22.000000      22.000000
mean          5.686364   78923.227273
std           2.914381   29324.289461
min           1.100000   37731.000000
25%           3.325000   56006.000000
50%           5.600000   74650.500000
75%           8.125000  104512.000000
max          10.500000  122391.000000
Training Score: 0.965147561174963
Testing Score: 0.9761526830926572
Coefficient: [[10052.26447106]]
Intercept: [21569.82824351]
```




```
Estimated Salary for 5 years of experience: 71831.1505988024
```

Experiment No. 18

Experiment to understand KNN algorithm for a given data set.

```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving Iris (1) - Iris (1).xlsx.csv to Iris (1) - Iris (1).xlsx.csv
```

```
import numpy as np
import pandas as pd
```

```
df = pd.read_csv('Iris (1) - Iris (1).xlsx.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
print(df.variety.value_counts())
```

```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

```
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 150,\n  \"fields\": [\n    {\n      \"column\": \"sepal.length\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n\"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\": 35,\n        \"samples\": [\n          6.2,\n          4.5,\n          5.6\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"sepal.width\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.435866284936698,\n        \"min\": 2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n\"samples\": [\n          2.3,\n          4.0,\n          3.5\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"petal.length\",\n\"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.7652982332594667,\n        \"min\": 1.0,\n        \"max\": 6.9,\n
```

\"num_unique_values\": 43,\n   \"samples\": [\n   6.7,\n
3.8,\n   3.7\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"petal.width\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.7622376689603465,\n   \"min\":
0.1,\n   \"max\": 2.5,\n   \"num_unique_values\": 22,\n
\"samples\": [\n   0.2,\n   1.2,\n   1.3\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"variety\",\n
\"properties\": {\n   \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n   \"samples\": [\n
\"Setosa\",\n   \"Versicolor\",\n   \"Virginica\"\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain, xtest, ytrain, ytest = train_test_split(
    features, label, test_size=0.2, random_state=1
)

model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)
```

```
KNeighborsClassifier()
```

```
print(model_KNN)
```

```
KNeighborsClassifier()
```

```
print(model_KNN.score(xtrain, ytrain))
print(model_KNN.score(xtest, ytest))
```

```
0.9583333333333334
1.0
```

```
from sklearn.metrics import confusion_matrix
confusion_matrix(label, model_KNN.predict(features))
```

```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  2, 48]])
```

```
from sklearn.metrics import classification_report
print(classification_report(label, model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support
```

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.94      0.95        50
   Virginica       0.94      0.96      0.95        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

Experiment No. 19

Experiment to understand Logistic Regression for a given data set.

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix,classification_report

X,y=make_classification(n_samples=400,n_features=2,n_informative=2,n_r
edundant=0,n_clusters_per_class=1,random_state=42)
df=pd.DataFrame(X,columns=["Age_Feature","Salary_Feature"])
df["Purchased"]=y
print(df.head())
features=df[["Age_Feature","Salary_Feature"]].values
label=df["Purchased"].values
x_train,x_test,y_train,y_test=train_test_split(features,label,test_siz
e=0.2,random_state=42)
model=LogisticRegression()
model.fit(x_train,y_train)
print("Training Score:",model.score(x_train,y_train))
print("Testing Score:",model.score(x_test,y_test))
predictions=model.predict(x_test)
print("Confusion Matrix:\n",confusion_matrix(y_test,predictions))
print("Classification Report:\
n",classification_report(y_test,predictions))
print("Predicted Purchase:",model.predict(np.array([[0.5,-0.5]]))[0])
```

```
   Age_Feature  Salary_Feature  Purchased
0     1.431868       -0.953095          0
1     0.323473       -1.833522          0
2     0.977841        1.501977          1
3     1.986525        0.602109          0
4     1.356117        0.280041          1
Training Score: 0.88125
Testing Score: 0.9
Confusion Matrix:
 [[34  7]
 [ 1 38]]
Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.83      0.89        41
           1       0.84      0.97      0.90        39

    accuracy                           0.90        80
   macro avg       0.91      0.90      0.90        80
weighted avg       0.91      0.90      0.90        80

Predicted Purchase: 0
```

Experiment No. 20

Experiment to understand K-means clustering algorithm for a given data set.

```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving Mall_Customers - Mall_Customers.xlsx.csv to Mall_Customers -
Mall_Customers.xlsx.csv
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

df = pd.read_csv('Mall_Customers - Mall_Customers.xlsx.csv')
df.info()
print(df.head())

sns.pairplot(df)

features = df.iloc[:, [3, 4]].values

from sklearn.cluster import KMeans
model = KMeans(n_clusters=5, random_state=42, n_init=10)
model.fit(features)
print(model)

Final = df.iloc[:, [3, 4]].copy()
Final['label'] = model.predict(features)
print(Final.head())

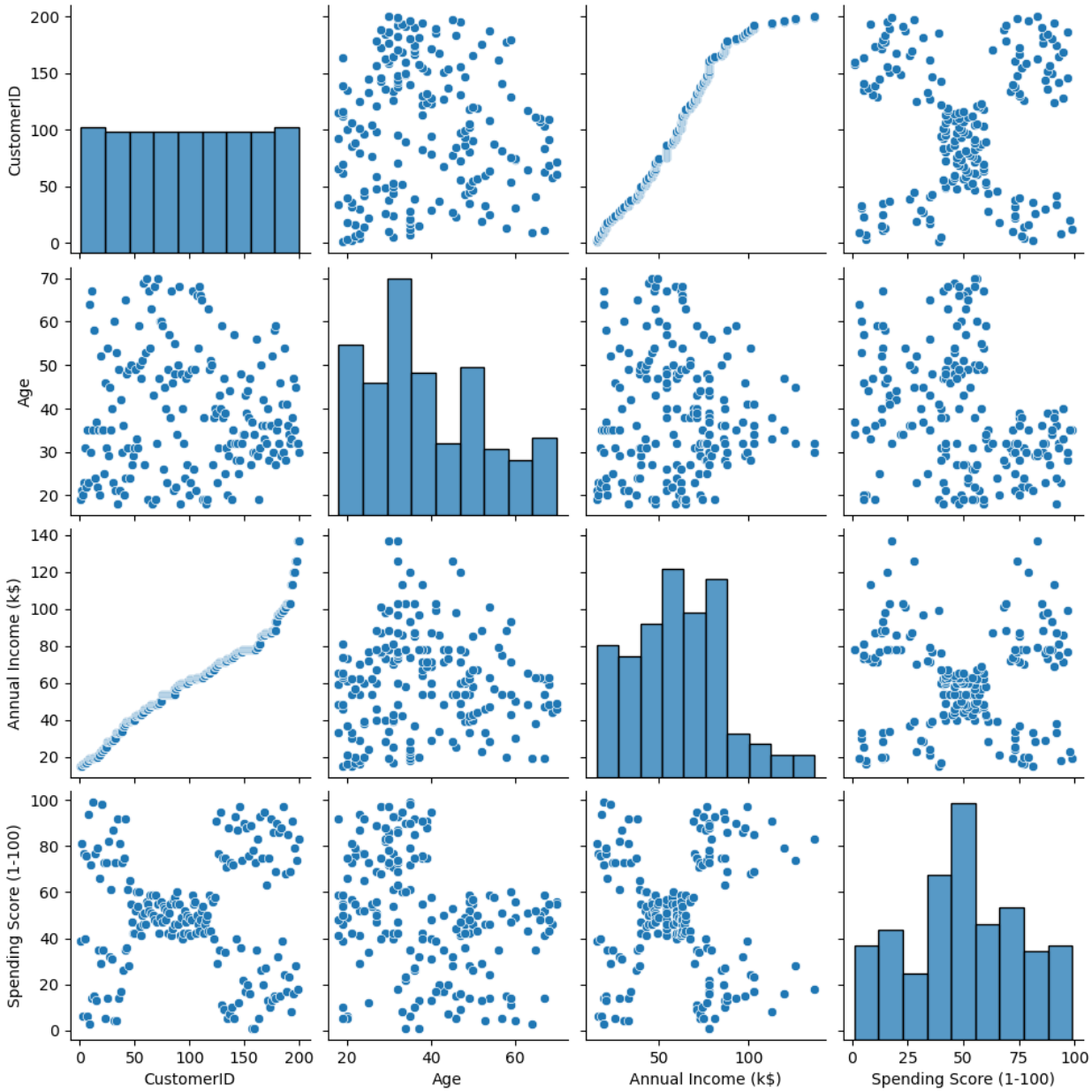
sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=8) \
    .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)")
\
    .add_legend()
plt.show()

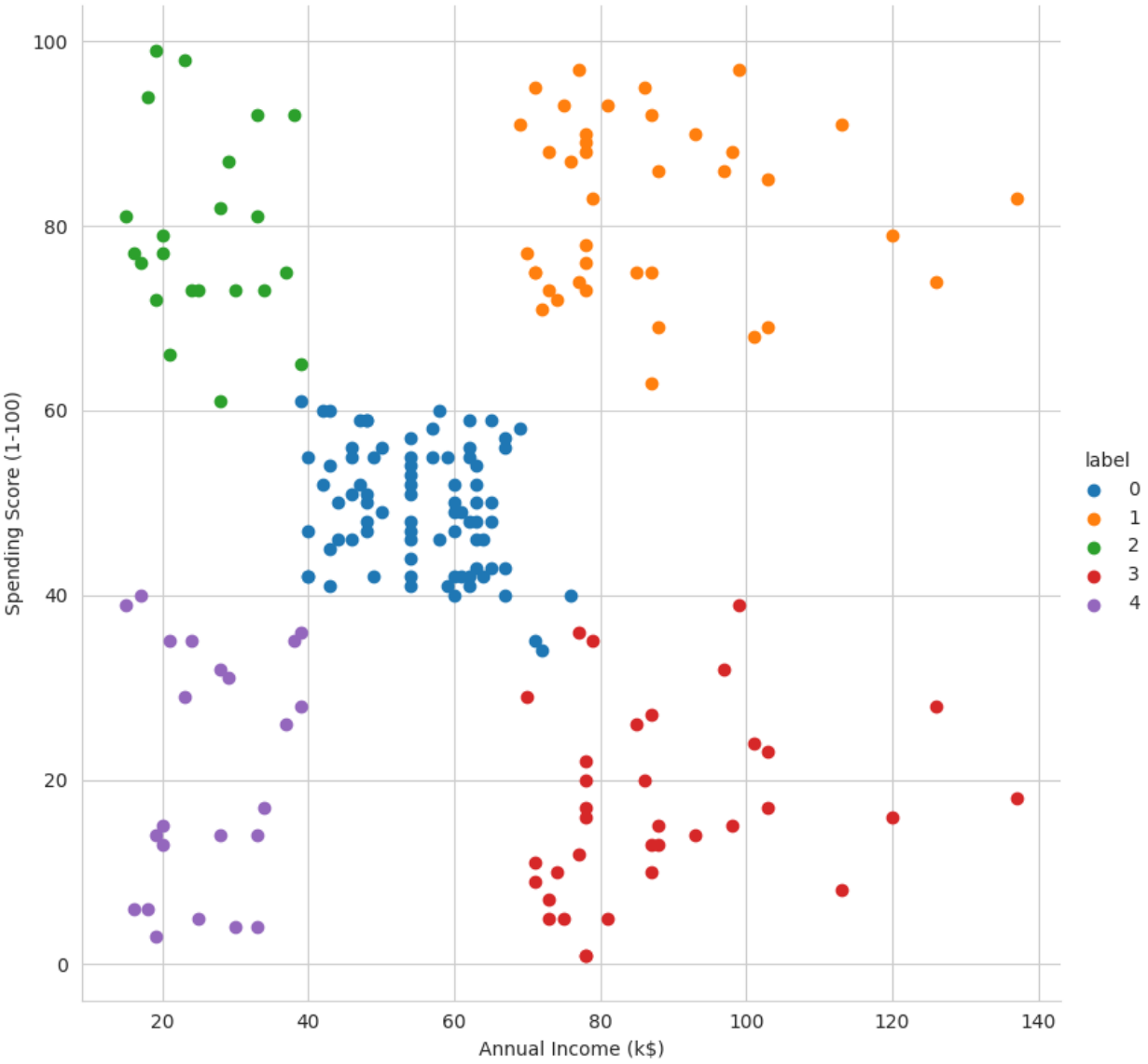
features_el = df.iloc[:, [2, 3, 4]].values
wcss = []

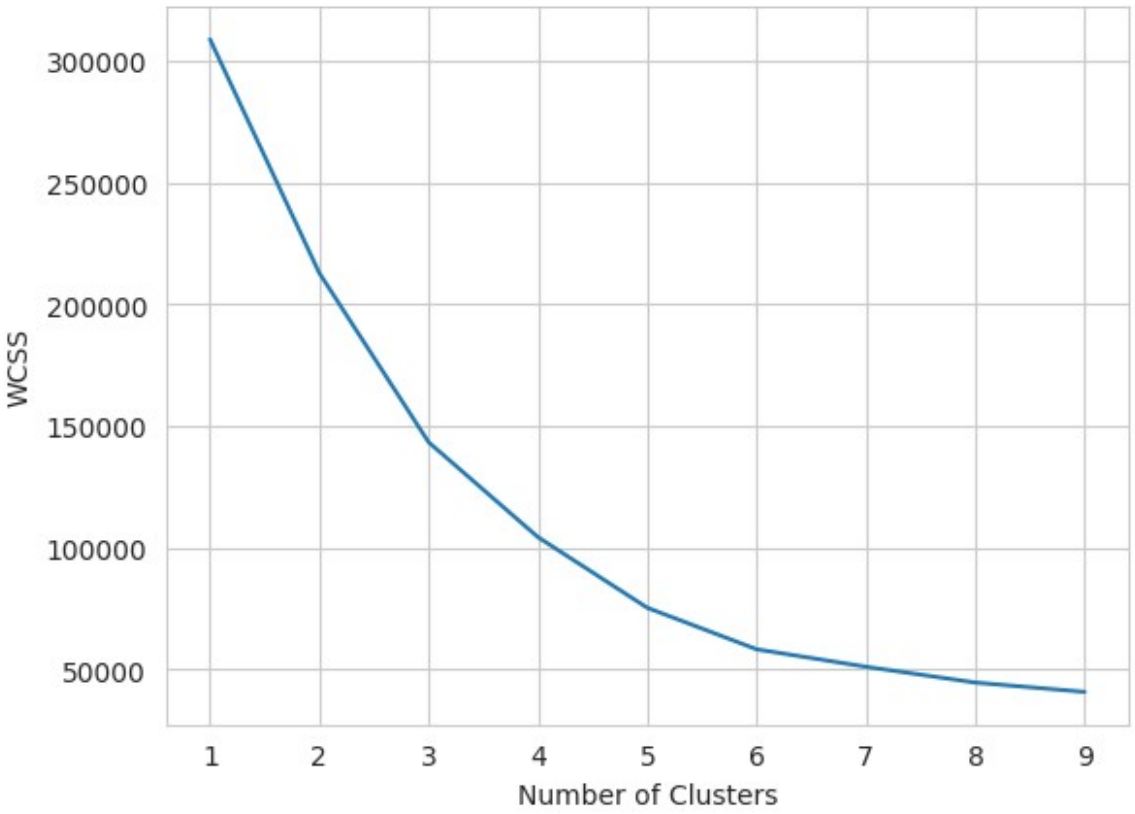
for i in range(1, 10):
    model = KMeans(n_clusters=i, random_state=42, n_init=10)
    model.fit(features_el)
    wcss.append(model.inertia_)

plt.plot(range(1, 10), wcss)
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
KMeans(n_clusters=5, n_init=10, random_state=42)
   Annual Income (k$)  Spending Score (1-100)  label
0                  15                      39      4
1                  15                      81      2
2                  16                       6      4
3                  16                      77      2
4                  17                      40      4
```


CustomerID
Age
Annual Income (k$)
Spending Score (1-100)
CustomerID
Age
Annual Income (k$)
Spending Score (1-100)


Spending Score (1-100)
Annual Income (k$)
label
0
1
2
3
4


300000
250000
200000
150000
100000
50000
WCSS
1
2
3
4
5
6
7
8
9
Number of Clusters